Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Matemática e Instituto de Cálculo

Métodos para interpretar ensambles de árboles

Tesis presentada para optar al título de Magister en Estadística Matemática de la Universidad de Buenos Aires.

Directora: Daniela Rodríguez
Autor: Martín Gabriel Cargnel

Abril, 2026

Resumen

Los ensambles de árboles, en particular *Random Forest* y *Gradient Boosting Machines*, son muy utilizados por su alta capacidad predictiva. Sin embargo, son considerados modelos de çaja negra"dado que por su complejidad resulta difícil el efecto de las covariables en la variable de respuesta. En este trabajo se estudiaron técnicas que permiten entender la importancia y el impacto que tienen las covariables, con el fin de utilizar modelos este tipo de modelo no solo para predecir, sino también para interpretar.

Índice general

1. Introducción **5**
1.1. Modelado Explicativo 5
1.2. Modelado Predictivo 5
1.3. *Trade-off* entre explicar y predecir 6
1.4. Aprendizaje automático interpretable 6
1.5. Ensambles de árboles como candidatos 7
1.6. Estructura del trabajo 7

2. CART **8**
2.1. Elementos que componen un árbol 8
2.2. Construyendo árboles de regresión 9
2.2.1. Criterio para particionar los datos 9
2.2.2. Definir cuándo un nodo es terminal 10
2.2.3. Asignar valores a nodos terminales 11
2.3. Poda de árboles 12
2.4. Limitaciones 12

3. Ensambles de árboles **14**
3.1. *Bagging* 14
3.1.1. Random Forest 17
3.2. Boosting 17
3.2.1. Algoritmo 18
3.2.2. Funciones de pérdida 19
3.2.3. Modelos base 20
3.2.4. Regularización 20

4. Interpretabilidad **23**
4.1. Importancia de las variables 23
4.1.1. *Split Gain Feature Importance* 23
4.1.2. *Permutation Feature Importance* 24
4.1.3. El problema de la inestabilidad: el *Rashomon Effect* 25
4.1.4. *Model Class Reliance*: una solución al *Rashomon Effect* 26
4.2. Efecto de las variables 28
4.2.1. *Partial Dependence Plots* (PDP) 28
4.2.2. *Individual Conditional Expectation* (ICE) 29

5. Aplicaciones **32**
5.1. *Airfoil Self-Noise* 32
5.2. *Concrete Compressive Strength* 35
5.3. *Wine Quality* 38

6. Conclusiones **41**

A. Apéndice **43**

1. Introducción

Este capítulo introduce los conceptos fundamentales del modelado estadístico, diferenciando entre los objetivos de predicción y explicación. Se analiza el compromiso o *trade-off* existente entre ambos y se propone el uso de técnicas de aprendizaje automático interpretable, específicamente en el contexto de ensambles de árboles. Finalmente, se detalla la estructura del trabajo y la notación utilizada.

Dentro del modelado estadístico se destacan dos objetivos principales: predecir y explicar. Esta distinción puede parecer filosófica en un primer momento, pero es de interés estudiarla dado que genera diferencias metodológicas importantes que llevan a los investigadores y practicantes a optar por técnicas y modelos muy distintos para alcanzar sus objetivos.

A continuación, se estudian las dos principales corrientes descritas en [Shmueli, 2010].

1.1. Modelado Explicativo

El modelado explicativo se centra en la comprensión de las relaciones subyacentes entre las variables, priorizando la interpretabilidad y el sustento teórico.

Cuando el interés es interpretar o explicar, los estudios se centran en entender el efecto que tienen variables explicativas (X) en una variable de respuesta (Y). Este tipo de estudios tratan de emplear modelos más simples para que sea más sencillo entender cómo cada covariable se relaciona con la variable de respuesta. En estos casos, típicamente se utilizan regresiones lineales por su simplicidad.

Generalmente esta clase de modelado se utiliza mucho en las ciencias sociales, donde existe una teoría muy fuerte por detrás que sustenta los modelos. En definitiva, se puede considerar que los modelos se utilizan con el objetivo de testear las teorías con datos empíricos, muchas veces intentando entender relaciones causales. Finalmente, pese a que la capacidad de predicción, medida utilizando alguna métrica como error cuadrático medio o error de clasificación, pasa a un segundo plano, es de interés encontrar modelos que superen un cierto umbral de precisión.

En resumen, el objetivo principal de este enfoque es validar teorías y cuantificar relaciones, aceptando limitaciones en la capacidad predictiva a cambio de transparencia en el modelo.

1.2. Modelado Predictivo

Por otro lado, el modelado predictivo se enfoca en la precisión de las estimaciones futuras, a menudo sacrificando la simplicidad del modelo.

Cuando el objetivo es predecir, el foco cambia y se busca estimar valores de la variable dependiente (Y) fuera de la muestra, dadas ciertas variables explicativas o predictoras (X). En estos casos se suelen utilizar modelos más complejos, como ensambles de árboles o redes neuronales, normalmente asociados al aprendizaje automático. El procesamiento del lenguaje natural es un caso claro donde el objetivo es predecir. Para esta disciplina se emplean redes neuronales con múltiples capas que hacen muy difícil su interpretación que, de todas formas, pasa a un segundo plano.

En conclusión, este paradigma prioriza la minimización del error de predicción utilizando algoritmos flexibles, donde la comprensión del mecanismo generador de datos es secundaria.

1.3. *Trade-off* entre explicar y predecir

Existe una tensión inherente entre la capacidad de un modelo para explicar la realidad y su capacidad para predecir eventos fuera de la muestra, conocida como el *trade-off* entre interpretabilidad y flexibilidad.

Según lo descrito anteriormente, parece haber un *trade-off* entre predecir y explicar. Es decir, que si el objetivo es predecir se debería optar por modelos más complejos, poco interpretables pero muy flexibles, para minimizar lo más posible una métrica de error. Por otra parte, si el objetivo es explicar, deberíamos aceptar modelos con baja capacidad predictiva, dado que para aumentar el poder predictivo necesariamente deberíamos complejizar demasiado los modelos, al punto de no poder interpretarlos.

Este *trade-off* también puede entenderse como que en aplicaciones donde se busca predecir, el modelo se adapta a los datos, mientras que cuando se busca explicar, los datos (o la realidad) se simplifican para ser explicados por el modelo. Por lo que es de interés estudiar técnicas que permitan utilizar modelos complejos para explicar el efecto de X sobre Y.

1.4. Aprendizaje automático interpretable

El aprendizaje automático interpretable surge como un campo de estudio destinado a dotar de transparencia a los modelos predictivos complejos.

Generalmente los modelos de aprendizaje automático destacan por su capacidad predictiva. Sin embargo, muchas veces se consideran cajas negras dado que, debido a su complejidad, es muy difícil responder preguntas como: ¿Qué variables son más importantes? o ¿Cuál es el efecto de las variables explicativas en la variable de respuesta?

Sin embargo, existe una batería de técnicas que permiten responder estas preguntas e intentar mitigar el *trade-off* descrito anteriormente. Esto se utiliza dado que la cantidad de modelos que pueden interpretarse "directamente" es limitada. Pero al usar estas técnicas se evita tener que restringir tanto la cantidad de modelos a utilizar, pudiendo emplear, por ejemplo, modelos más flexibles.

En definitiva, estas metodologías permiten aprovechar el alto rendimiento de los modelos de caja negra sin renunciar a la comprensión de su comportamiento.

1.5. Ensambles de árboles como candidatos

Dentro de los modelos de aprendizaje automático, los ensambles de árboles presentan características favorables para balancear precisión e interpretabilidad.

En este trabajo se decidió estudiar los ensambles de árboles como modelo ya que, junto con técnicas de interpretabilidad, permiten mitigar el *trade-off* entre interpretabilidad y poder predictivo. Este tipo de modelo se eligió sobre otras alternativas debido a su alto poder predictivo, lo que lo hace muy popular para tareas de predicción. Su popularidad y buen rendimiento resultaron en la disponibilidad de numerosos paquetes en varios lenguajes de programación, lo que facilita su implementación de manera sencilla y eficiente. Además, dado que los ensambles de árboles se basan en utilizar múltiples árboles de decisión para la predicción, mantienen algunas ventajas de los árboles individuales, como la capacidad de modelar las interacciones entre las variables.

Cabe destacar que si bien estos modelos pueden utilizarse tanto para clasificación como para regresión, en este trabajo solo se estudiará la metodología para problemas de regresión. De todas formas, todos los resultados a los que se lleguen son fácilmente adaptables a problemas de clasificación.

1.6. Estructura del trabajo

El trabajo se estructura de la siguiente manera: en el Capítulo 2 se hace una revisión de la teoría detrás de los árboles de regresión, mencionando el algoritmo que se usa para podarlos, cómo se comportan cuando las variables están correlacionadas y ante la presencia de *outliers*. Se finaliza con algunas de las limitaciones que llevaron al desarrollo de técnicas más avanzadas. En el Capítulo 3 se estudian los ensambles de árboles y cómo estos alivian algunas de las limitaciones de los árboles individuales. En particular se profundiza sobre las técnicas *Bagging* y *Boosting*. Luego, en el Capítulo 4, se revisan tres técnicas que permiten interpretar los ensambles de árboles descritos anteriormente. Finalmente, se presentan 3 aplicaciones de todo lo descrito anteriormente en el Capítulo 5, seguido por las conclusiones en el Capítulo 6.

2. CART

En este capítulo se hará una revisión sobre árboles de regresión y clasificación (CART), basando en [Breiman et al., 1984].

Estos modelos entran dentro de la categoría de análisis supervisado no paramétrico, por lo que tienen una gran flexibilidad que les permite modelar naturalmente las interacciones entre variables y no asumen linealidad. Otra de las ventajas de estos algoritmos es su interpretabilidad, dado que es muy sencillo entender el efecto de cada variable explicativa en la variable dependiente siguiendo la estructura del árbol.

Con el objetivo de introducir el modelo estadístico en cuestión, en la Figura 2.1 se muestra un árbol de decisión cuyo objetivo es predecir el número de anillos de un abulón, el cual es un indicador de su edad[1]. El árbol se interpreta de arriba hacia abajo, lo que indica que el peso de la concha (*Shell_weight*) es la variable más relevante, ya que aparece en la primera división. Se observa que valores más altos de *Shell_weight* están asociados con un mayor número de anillos.

Por otro lado, el peso de la carne extraída (*Shucked_weight*) también influye en la variable dependiente, pero su efecto se observa después de las primeras condiciones basadas en el peso de la concha. Por ejemplo, para abulones con un *Shell_weight* mayor o igual a 0.37 y un *Shucked_weight* menor o igual a 0.55, el número de anillos predicho es 14.85.

Con este breve ejemplo se puede apreciar la facilidad con la que se pueden interpretar visualmente los árboles de decisión y cómo estos modelos representan de manera natural las interacciones entre diferentes variables. A continuación, se explicarán los elementos que componen un árbol de decisión y cómo se construyen.

2.1. Elementos que componen un árbol

Antes de pasar a la construcción de los árboles, es importante definir los elementos que los componen. En primer lugar definimos los nodos como los puntos en los cuales se dividen las observaciones. A su vez pueden clasificarse como: la raíz, que es el primer nodo que contiene todas las observaciones y sobre el cual se hace la primera partición; los nodos internos, que son los puntos en los cuales se dividieron las observaciones; y los nodos terminales u hojas, que presentan las predicciones finales del modelo y no tienen descendientes inmediatos.

Luego, como se ve en la Figura 2.1, los nodos definidos anteriormente están unidos por flechas que llamaremos ramas y tienen el objetivo de conectar los distintos nodos del árbol.

[1] Los datos provienen de [Nash et al., 1994]

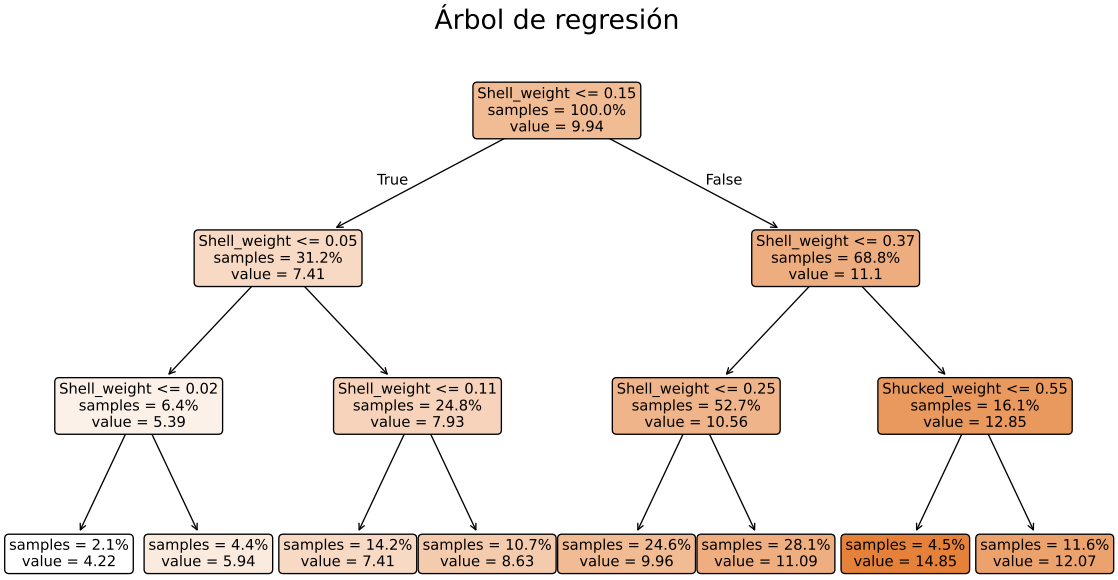

Figura 2.1: Árbol de decisión para predecir el número de anillos de un abulón utilizando medidas físicas como el peso de la concha y el peso de la carne extraída.

Habiendo definido estos elementos podemos pasar a estudiar cómo construir los árboles.

2.2. Construyendo árboles de regresión

Para construir los árboles necesitamos particionar secuencialmente el conjunto de datos X que tenemos. El objetivo final es construir un modelo de regresión que nos permita estimar el valor de una variable dependiente Y. Para ello necesitamos tres elementos: un criterio para particionar los datos, una regla para determinar cuándo un nodo es terminal y una forma de asignar un valor a las predicciones de los nodos terminales.

2.2.1. Criterio para particionar los datos

Dado que necesitamos crear $R_1, R_2, ..., R_J$ regiones distintas y disjuntas, éstas podrían tener a priori cualquier forma. Sin embargo, por simplicidad, asumiremos que tienen forma rectangular. Además, dado que se trata de un algoritmo de regresión supervisado tiene el objetivo de predecir con el menor error posible, por lo que se suele utilizar el error cuadrático medio para medir el error:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 \tag{2.1}$$

con n la cantidad de observaciones, Y los valores reales de la variable dependiente y $\hat{Y}$ las predicciones realizadas por nuestro modelo con la muestra X.

En definitiva, si lo que buscamos son regiones que minimicen el error cuadrático medio podemos expresarlo como

$$MSE(T) = \frac{1}{n} \sum_{j=1}^{J} \sum_{i \in R_j} (Y_i - \hat{Y}_{R_j})^2 \tag{2.2}$$

con el árbol T, n el número total de observaciones en la muestra, J el número de regiones R, Y_i el valor real de las observaciones i que se encuentran en la región R_j y $\hat{Y}_{R_j}$ la predicción de nuestro modelo.

Sin embargo, calcular esta expresión es computacionalmente prohibitivo dado que implica buscar todas las posibles particiones en todas las posibles regiones que se pueden generar con la muestra. A modo de ejemplo, esto significa que si tenemos p variables y cada variable tiene s puntos en los que podría ser particionada, el número total de posibles particiones de todas las variables en cada nodo sería $s \cdot p$. Dado que cada partición es binaria, el número de nodos crece exponencialmente con la profundidad del árbol d. Es decir que para calcular la cantidad total de particiones posibles (TP) para todo el árbol tendríamos que calcular

$$TP = \sum_{d=0}^{D} s \cdot p \cdot 2^d \tag{2.3}$$

que crece exponencialmente por lo que se hace computacionalmente muy intensiva.

Es por ello que se utiliza el algoritmo conocido como división binaria recursiva, este algoritmo, que va de arriba hacia abajo en el árbol, se considera *greedy* dado que en cada paso solo mira el valor óptimo en ese momento, por lo que hace menos cálculos aunque puede que no llegue a la solución óptima. En definitiva el algoritmo consiste en considerar todas las variables $X_1, X_2, ..., X_p$ y todos los posibles valores en los cuales se podría particionar (s) para quedarnos con el que minimice el error cuadrático medio en ese paso. En definitiva se busca

$$MSE(T) = \frac{1}{n} \sum_{r=1}^{R} \sum_{i:x_i \in R_r(j,s)} (Y_i - \hat{Y}_{R_r})^2 \tag{2.4}$$

con n el número de observaciones, R el número total de regiones después de aplicar todas las divisiones, $R_r(j, s)$ es la región r creada al dividir el predictor X_j en el punto de corte s, siendo X_j el predictor que minimiza el error, Y_i el valor real de las observaciones i dentro de la región R_r y $\hat{Y}_{R_r}$ los valores predichos para todas las observaciones i en la región R_r.

Este algoritmo ya no crece de manera exponencial como el anterior, por lo que es más sencillo de calcular.

2.2.2. Definir cuándo un nodo es terminal

Definir cuándo un nodo es terminal es equivalente a establecer una regla de parada en el algoritmo de partición recursiva. En teoría, se podría continuar dividiendo los nodos hasta que cada uno contenga una única observación, lo cual minimizaría el error en la muestra de entrenamiento a cero. No obstante, esto conduciría a un modelo sobreajustado con poca capacidad de predicción en datos fuera de la muestra.

Para evitar esto, se puede definir un nodo como terminal cuando se cumple alguna condición restrictiva. Las condiciones más habituales incluyen que el número de observaciones en el nodo sea menor a un mínimo preestablecido (por ejemplo, $n_{min} = 5$) o que la disminución del error cuadrático medio que se obtendría con la mejor partición posible no supere un cierto umbral. También se detiene el proceso si el nodo es puro, es decir, si todas las observaciones tienen el mismo valor de respuesta.

Es importante notar que detener el crecimiento del árbol demasiado pronto puede impedir que se descubran estructuras importantes en los datos que solo aparecerían tras particiones adicionales. Por esta razón, la estrategia recomendada y más utilizada consiste en establecer criterios de parada laxos para construir un árbol inicialmente muy grande (T_0), y posteriormente aplicar técnicas de poda (*pruning*) para encontrar el subárbol óptimo, como se explicará más adelante.

2.2.3. Asignar valores a nodos terminales

Dado que la función a optimizar es el error cuadrático medio, se puede demostrar que la media muestral será la constante que minimice ese error.

Partimos de que queremos encontrar el valor de c que minimice

$$MSE(c) = \frac{1}{n}\sum_{i=1}^{n}(y_i - c)^2 \tag{2.5}$$

para ello primero derivamos con respecto de c

$$\begin{aligned}
\frac{d}{dc}MSE(c) &= \frac{d}{dc}\left(\frac{1}{n}\sum_{i=1}^{n}(y_i - c)^2\right) \\
&= \frac{1}{n}\sum_{i=1}^{n}\frac{d}{dc}\left[(y_i - c)^2\right] \\
&= \frac{1}{n}\sum_{i=1}^{n}2(y_i - c)(-1) \\
&= -\frac{2}{n}\sum_{i=1}^{n}(y_i - c)
\end{aligned} \tag{2.6}$$

y luego igualamos a cero:

$$\begin{aligned}
-\frac{2}{n}\sum_{i=1}^{n}(y_i - c) &= 0 \\
\frac{1}{n}\sum_{i=1}^{n}y_i &= c
\end{aligned} \tag{2.7}$$

Por lo que los valores que tendrán los nodos terminales (predicciones) serán simplemente la media de las observaciones que están en ese nodo.

2.3. Poda de árboles

Para evitar el sobreajuste, la técnica que se estudiará se basa en hacer crecer un árbol muy grande y luego "podarlo" para obtener árboles más pequeños que podemos comparar con validación cruzada. Esta última es una técnica que permite estimar el error de generalización dividiendo el conjunto de datos en k grupos o *folds*. El procedimiento consiste en entrenar el modelo utilizando $k-1$ grupos y evaluarlo en el grupo restante, repitiendo este proceso k veces para que cada grupo funcione como conjunto de prueba una vez. El error de validación final es el promedio de los errores obtenidos en cada iteración, lo que permite seleccionar el parámetro de complejidad que minimiza dicho error.

Antes de describir el procedimiento es importante definir T_0 como un árbol maximal y considerar que T' sea un subárbol podado de T. Para que un árbol sea considerado subárbol de otro tiene que tener $T' = T_0 - T_t$, donde T_t es una rama (subárbol) que se elimina de T_0 para obtener T'.

El problema de querer encontrar subárboles es que pueden ser demasiados, haciendo que sea computacionalmente muy exigente. Es por ello que se utiliza *cost complexity pruning* para trabajar un pequeño número de subárboles. A continuación vemos la ecuación:

$$\sum_{m=1}^{|T|} \sum_{i \in R_m} (Y_i - \hat{Y}_{R_m})^2 + \alpha |T| \tag{2.8}$$

con α que sirve para regular la intensidad de la regularización y $|T|$ el número de nodos terminales del árbol T. Esta expresión permite obtener una secuencia ordenada de subárboles que podemos evaluar por medio de validación cruzada. Queda claro que al aumentar α se generan árboles más pequeños.

Se puede ver en la Figura 2.2 como a medida que aumentan los valores de α inicialmente el error cuadrático medio disminuye, pero luego empieza a subir debido al sobre ajuste.

2.4. Limitaciones

Dado que este trabajo se centra en la interpretación de modelos, la principal limitación que se destaca es la falta de estabilidad (o alta varianza). Esto significa que pequeños cambios en los datos de entrenamiento pueden dar lugar a particiones muy diferentes y, por ende, a una estructura de árbol completamente distinta. Esta inestabilidad surge de la naturaleza jerárquica del algoritmo: un error o cambio en una división superior se propaga a todas las ramas inferiores [James et al., 2023]. Esto representa un desafío para la interpretación, ya que la importancia de las variables puede variar drásticamente con ligeras modificaciones en la muestra.

Otra limitación relevante es la dificultad que presentan los árboles para capturar estructuras aditivas simples. Mientras que una regresión lineal puede modelar eficientemente una relación del tipo $Y = \beta_0 + \sum \beta_j X_j + \epsilon$, un árbol de decisión necesita realizar múltiples particiones para aproximar esta estructura lineal mediante una función escalonada. Esto suele resultar en una menor capacidad predictiva en comparación con modelos paramétricos cuando la verdadera relación es lineal.

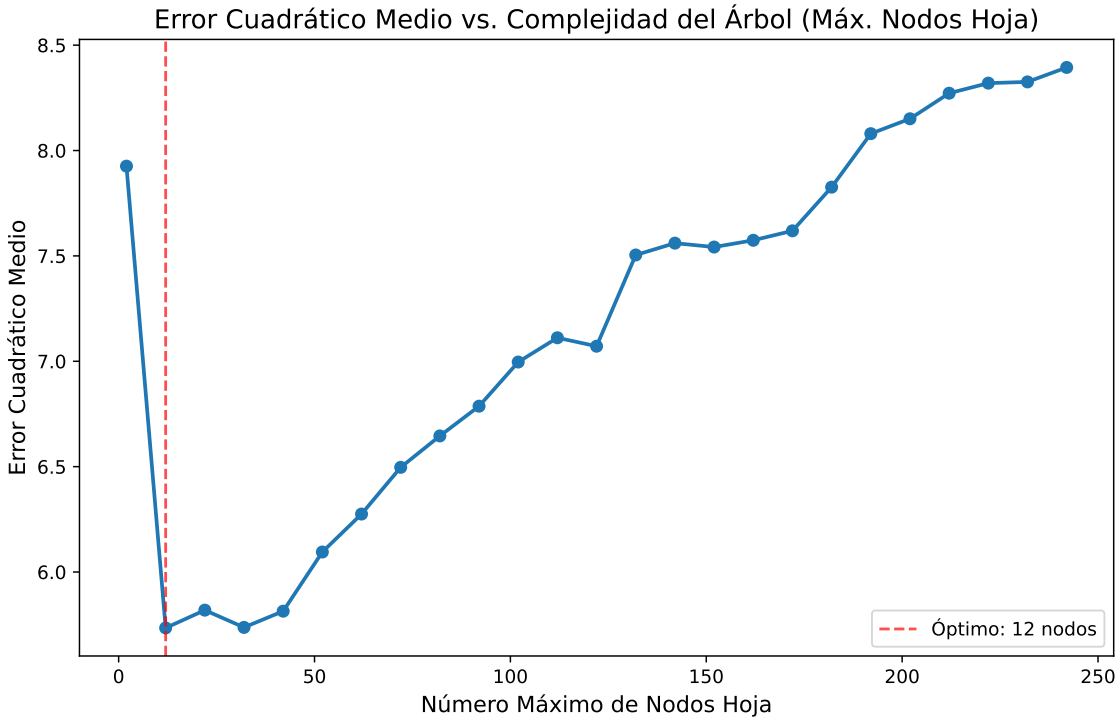

Figura 2.2: Error cuadrático medio vs complejidad del árbol para diferentes valores de α.

Además, las predicciones de los árboles de decisión no son continuas ni suaves, sino constantes por regiones. Esto puede ser una desventaja en problemas donde se espera que la variable respuesta cambie gradualmente con las variables explicativas.

Por último, en la práctica los árboles no suelen tener una alta capacidad predictiva, lo que limita su uso en problemas complejos y requiere el uso de técnicas como *bagging* o *boosting* para mejorar su capacidad predictiva y reducir la varianza.

3. Ensambles de árboles

Como se mencionó anteriormente, dos de los problemas principales de los árboles de decisión son la baja capacidad predictiva y la inestabilidad. Por lo que en este capítulo se estudian métodos para evitar estos problemas, en particular *boosting* y *bagging* con dos implementaciones clásicas como *Random Forest* y *Gradient Boosting Machines*. Estas técnicas mejoran la capacidad predictiva y reducen la variabilidad al agrupar múltiples modelos con el objetivo de tener un único modelo con mejor capacidad predictiva.

3.1. *Bagging*

Uno de los grandes problemas de los árboles decisión es la varianza que tienen, por lo que parece intuitivo aplicar técnicas que permitan promediar métodos estadísticos para reducir la varianza manteniendo el sesgo bajo.

La idea de *Bagging* [Breiman, 1996] es calcular $\hat{f}^1(x), \hat{f}^2(x), ..., \hat{f}^B(x)$ usando B muestras *bootstrap*[1] del conjunto de observaciones $(X, Y) = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$, y promediar las predicciones para reducir la varianza. Es decir

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^b(x) \tag{3.1}$$

La ventaja de utilizar *Bagging* para predecir es que no hay que preocuparse por la profundidad de los árboles, dado que los árboles individuales tendrán muy poco sesgo y la varianza será reducida al tomar promedios. Además, si bien *Bagging* puede utilizarse en múltiples métodos estadísticos, es particularmente útil para árboles de decisión dado que permite reducir la varianza sin aumentar el sesgo. Como se ve más formalmente a continuación, partiendo de las predicciones

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^b(x) \tag{3.2}$$

y asumiendo que los datos provienen de un modelo de la forma $Y = f(x) + \epsilon$, donde $f(x) = E[Y \mid x]$ es la verdadera función que relaciona las covariables con la respuesta y ϵ es un término de error con $E[\epsilon] = 0$ y $Var(\epsilon) = \sigma^2$, el error cuadrático medio de la predicción se puede descomponer como

[1] El bootstrap es un método de remuestreo que se utiliza para cuantificar la incertidumbre o reducir la varianza de un estimador. Consiste en generar múltiples muestras simuladas a partir de un único conjunto de datos mediante extracciones aleatorias con reemplazo. Al utilizar el reemplazo, cada nueva muestra mantiene el tamaño del conjunto original, pero altera la distribución al repetir ciertas observaciones y excluir otras.

$$\begin{aligned} MSE &= E\Big[\big(Y - \hat{f}_{bag}(x)\big)^2\Big] \\ &= \Big(E[\hat{f}_{bag}(x)] - f(x)\Big)^2 + Var\Big(\hat{f}_{bag}(x)\Big) + \sigma^2 \end{aligned} \tag{3.3}$$

El primer término, $\big(E[\hat{f}_{bag}(x)] - f(x)\big)^2$, es el sesgo al cuadrado y mide cuánto se desvía, en promedio, la predicción del modelo respecto de la verdadera función $f(x)$. El segundo término, $Var(\hat{f}_{bag}(x))$, es la varianza del modelo y captura cuánto cambian las predicciones al variar la muestra de entrenamiento. El tercer término, $\sigma^2 = Var(Y - f(x))$, es el error irreducible, es decir, la variabilidad intrínseca de los datos que ningún modelo puede capturar. Dado que *Bagging* utiliza árboles de mucha profundidad, el sesgo se mantiene bajo, por lo que la mejora proviene principalmente de la reducción de la varianza. Concentrándose entonces en cómo la varianza se ve afectada

$$Var\Big(\hat{f}_{bag}(x)\Big) = Var\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)\right) = \frac{1}{B^2}Var\left(\sum_{b=1}^{B}\hat{f}^b(x)\right) \tag{3.4}$$

asumiendo que los $\hat{f}^b(x)$ están idénticamente distribuidos, se puede reescribir:

$$Var\left(\sum_{b=1}^{B}\hat{f}^b(x)\right) = \sum_{b=1}^{B}Var\Big(\hat{f}^b(x)\Big) + \sum_{i=1}^{B}\sum_{j\neq i}^{B}cov\Big(\hat{f}^i(x), \hat{f}^j(x)\Big) \tag{3.5}$$

ahora asumiendo varianzas iguales ($Var(\hat{f}^b(x)) = \sigma^2$) y correlaciones idénticas (ρ) para cada par de árboles:

$$Var\left(\sum_{b=1}^{B}\hat{f}^b(x)\right) = B\sigma^2 + B(B-1)\rho\sigma^2 \tag{3.6}$$

volviendo a la varianza total:

$$\begin{aligned} Var\Big(\hat{f}_{bag}(x)\Big) &= \frac{1}{B^2}\Big(B\sigma^2 + B(B-1)\rho\sigma^2\Big) \\ &= \sigma^2\left(\frac{1}{B} + \frac{\rho B(B-1)}{B^2}\right) \end{aligned} \tag{3.7}$$

Por lo tanto, cuando $B \to \infty$, el término $\frac{1}{B} \to 0$ y la varianza se aproxima a $\rho\sigma^2$. Esto implica que la reducción de la varianza depende tanto de B como de ρ. Dado que la correlación entre árboles ρ es menor a 1, la varianza final ($\rho\sigma^2$) resultará menor que la varianza original (σ^2), demostrando la efectividad del método.

Bagging tiene una forma muy natural de calcular el error, básicamente se basa en el hecho de que cuando se hace una muestra *bootstrap* solamente se toma en cuenta aproximadamente el 63 % de los datos. Para demostrarlo, partimos de la probabilidad de no ser seleccionado en una muestra:

$$P(\text{No ser seleccionado en una muestra}) = 1 - \frac{1}{n} \tag{3.8}$$

Luego, como se realizan n extracciones con reemplazo, por lo que la probabilidad de no ser seleccionado en ninguna muestra es

$$P(\text{No ser seleccionado en ninguna muestra}) = \left(1 - \frac{1}{n}\right)^n \tag{3.9}$$

cuando $n \to \infty$:

$$\lim_{n\to\infty} \left(1 - \frac{1}{n}\right)^n = \frac{1}{e} \approx 0{,}368 \tag{3.10}$$

Esto significa que la probabilidad de ser seleccionado al menos una vez es $1 - \frac{1}{e} \approx 0{,}632$. Es decir, aproximadamente el 63.2 % de las observaciones serán incluidas en cada muestra *bootstrap*, mientras que el restante 36.8 % de las observaciones quedarán fuera de la muestra (*out-of-bag*).

Este resultado permite estimar el error usando las observaciones *out-of-bag* como muestra de testeo. Con un B suficientemente grande, este error es asintóticamente equivalente a *leave-one-out cross validation*, lo cual es particularmente útil cuando se cuenta con una muestra grande donde el *leave-one-out cross validation* tradicional sería computacionalmente costoso. Esta relación se evidencia en la Figura 3.1, que compara las estimaciones de error *out-of-bag* y *leave-one-out cross validation* utilizando el conjunto de datos *Abalone* y variando la cantidad de árboles B. Se observa claramente que, al incrementar el número de árboles, ambas métricas convergen hacia valores similares.

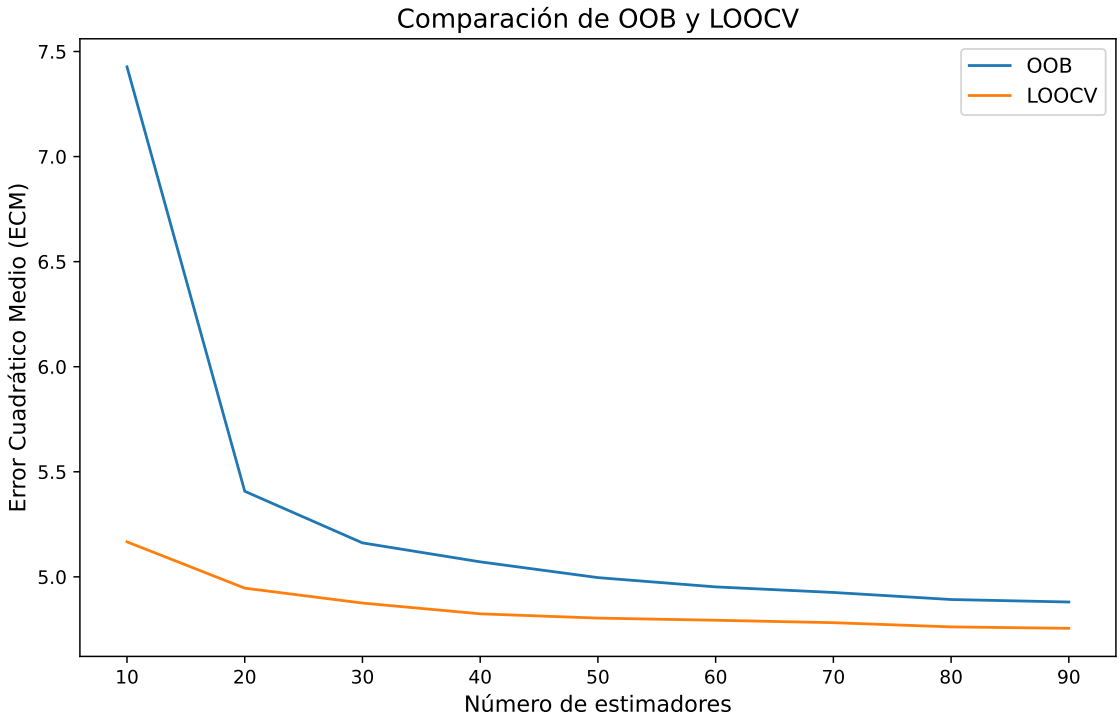

Figura 3.1: Comparación entre el error de *out-of-bag* y *leave-one-out cross validation* para modelos de *Bagging* con distintos números de árboles ajustados al conjunto de *Abalone*.

Una de las limitaciones de *Bagging* es que, al utilizar todas las variables independientes en cada árbol del modelo, puede no reducir significativamente la varianza cuando estas variables están altamente correlacionadas. Esto se debe a que los árboles generados tienden a ser similares, lo que limita el beneficio del promediado de las predicciones. Este punto puede verse en la demostración anterior, donde a menor ρ menor es el error.

3.1.1. Random Forest

Random Forest [Breiman, 2001a] propone una mejora con respecto a *Bagging* al tener en cuenta solo un sub-conjunto de las covariables para cada *split*. Esto permite que los árboles sean distintos entre ellos, haciendo que estén menos correlacionados y por lo tanto reduciendo más el error.

El parámetro que define la cantidad de variables que se tiene cuenta en cada *split* se denomina m. Cabe destacar que un *Random Forest* con $m = p$, siendo p la cantidad de covariables, es *Bagging*. Por lo que *Random Forest* puede considerarse un caso particular de *Bagging*.

Más formalmente se puede pensar el predictor *Random Forest* como

$$\hat{f}_{\text{rf}}(x) = \frac{1}{B}\sum_{b=1}^{B} T(x;\Theta_b) \tag{3.11}$$

donde $(T(x;\Theta_b))$ representa el valor predicho por el b-ésimo árbol de decisión para una observación x, construido a partir de una muestra *bootstrap* y parámetros aleatorios Θ_b.

Es interesante notar que, si tomamos si $B \to \infty$, es decir cuando se promedian muchos árboles, por Ley de los Grandes Números se puede probar que:

$$\lim_{B\to\infty} \hat{f}_{\text{rf}}(x) = E[T(x;\Theta_b)] \tag{3.12}$$

Esto significa que, con suficientes árboles, la predicción de *Random Forest* se aproxima al valor esperado de un único árbol. La demostración formal y los detalles pueden verse en [Breiman, 2001a]. Sin embargo, de manera intuitiva puede verse en la Figura 3.2 cómo a medida que aumenta la cantidad de árboles la variabilidad de las predicciones disminuye.

3.2. Boosting

A diferencia de *Bagging*, *Boosting* no promedia predicciones, sino que construye un modelo iterativamente, donde cada modelo se construye con el objetivo de corregir el error cometido por el modelo anterior. A continuación se presenta, para el caso de regresión, un ejemplo de cómo funciona el algoritmo de *Gradient Boosting Machines* (*GBM*) propuesto por [Friedman, 2001].

Como en cualquier problema de aprendizaje supervisado, el objetivo es estimar $\hat{f}(x)$ a partir de un conjunto de datos (X, Y). En este caso, el conjunto de datos es una muestra i.i.d. de (X, Y) de tamaño n:

$$(X, Y) = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\} \tag{3.13}$$

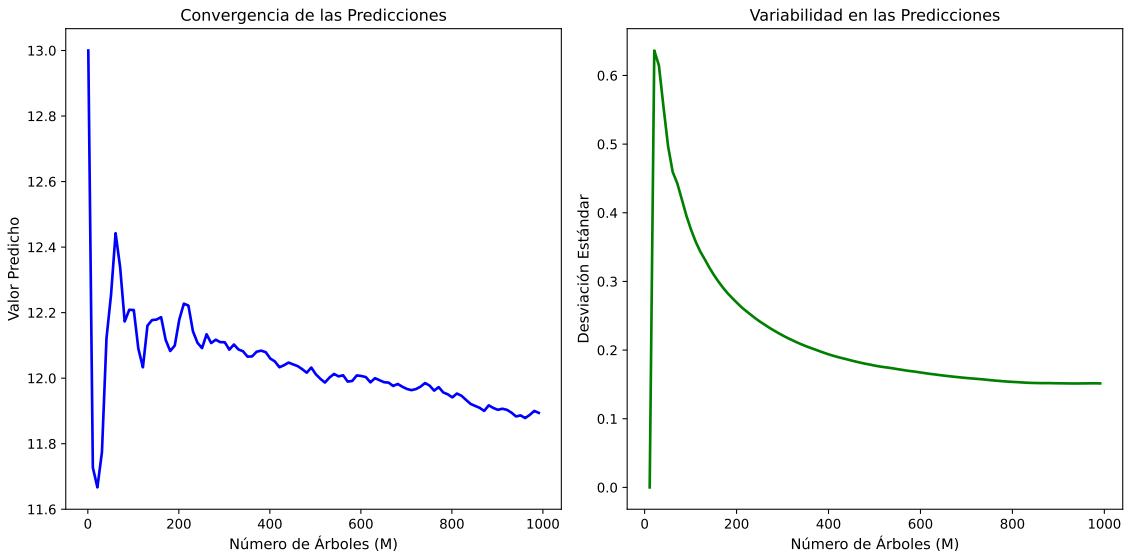

Figura 3.2: Convergencia y variabilidad de las predicciones para modelos de Random Forest con distintos números de árboles.

donde x_i es un vector de covariables y y_i es la variable de respuesta continua.

El algoritmo de *Gradient Boosting Machine* propone un método para encontrar un aproximación de la función $f(x)$ que minimiza una función de pérdida $L(y, f(x))$:

$$\hat{f}(x) = \arg\min_{f} \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(x_i)) \tag{3.14}$$

En definitiva, se ve que la forma de aproximar la función $f(x)$ es a través de una combinación lineal de modelos base $h(x, \theta)$:

$$\hat{f}(x) = \sum_{b=1}^{B} \gamma_b h_b(x, \theta_b) \tag{3.15}$$

3.2.1. Algoritmo

El algoritmo de *Gradient Boosting Machine* requiere un conjunto de datos (X, Y), una función de pérdida $L(y, f(x))$, un número de iteraciones B y un modelo base $h(x, \theta)$. Con esto, el algoritmo es el siguiente:

1. Iniciar $\hat{f}_0(x)$ con una constante.

2. Para cada iteración $b = 1, 2, ..., B$:

   - Calcular el negativo del gradiente de la función de pérdida para cada observación $i = 1, ..., n$:

$$-g_{ib} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right] \tag{3.16}$$

   - Ajustar el modelo base $h_b(x, \theta_b)$ a los valores $-g_{ib}$.

- Encontrar el mejor tamaño de paso γ_b:

$$\gamma_b = \arg\min_{\gamma} \sum_{i=1}^{n} L\Big(y_i, \hat{f}_{b-1}(x_i) + \gamma h_b(x_i, \theta_b)\Big) \tag{3.17}$$

- Actualizar la aproximación:

$$\hat{f}_b(x) = \hat{f}_{b-1}(x) + \gamma_b h_b(x, \theta_b) \tag{3.18}$$

3. Devolver el modelo final $\hat{f}_B(x)$.

3.2.2. Funciones de pérdida

Dependiendo del problema y los datos con los que se esté trabajando, se pueden utilizar distintas funciones de pérdida. A continuación se presentan algunas de las más comunes para problemas de regresión.

Error Cuadrático Medio (MSE)

Una de las funciones de pérdida más comunes es el error cuadrático medio (MSE), que se define como:

$$L(y, f(x)) = \frac{1}{2}(y - f(x))^2 \tag{3.19}$$

Una de las ventajas que tienen este tipo de funciones de pérdida es que son diferenciables, lo cual facilita el cálculo del gradiente. Que simplemente es:

$$\begin{aligned} \frac{\partial L(y, f(x))}{\partial f(x)} &= \frac{\partial}{\partial f(x)} \frac{1}{2}(y - f(x))^2 \\ &= \frac{1}{2} 2(y - f(x)) \frac{\partial (y - f(x))}{\partial f(x)} \\ &= -(y - f(x)) = f(x) - y \end{aligned} \tag{3.20}$$

Luego, como el gradiente indica la dirección de máximo crecimiento, el negativo del gradiente indica la dirección de máximo decrecimiento. Lo cual es lo que se busca en el algoritmo de *boosting*, es decir, minimizar la función de pérdida. Por lo que el negativo del gradiente evaluado para la observación i en la iteración b resulta ser:

$$-g_{ib} = y_i - \hat{f}_{b-1}(x_i) \tag{3.21}$$

es decir, el residuo.

Alternativas robustas

Como se mencionó anteriormente, una de las ventajas de utilizar la función de pérdida MSE es que es diferenciable. Sin embargo, el hecho de que penalice los errores grandes puede ser una desventaja cuando se trabaja con *outliers*. Por lo que también se puede utilizar la función de pérdida absoluta (MAE) que se define como:

$$L(y, f(x)) = |y - f(x)| \tag{3.22}$$

ó la función de pérdida Huber que se define como:

$$L(y, f(x)) = \begin{cases} (y - f(x))^2 & \text{si } |y - f(x)| \leq \delta \\ 2\delta|y - f(x)| - \delta^2 & \text{si } |y - f(x)| > \delta \end{cases} \tag{3.23}$$

A continuación, en la Figura 3.3, se presentan las funciones de pérdida anteriores. Puede verse que MSE aumenta mucho cuando hay errores grandes, mientras que MAE y Huber son más robustas a los *outliers*. Además, Huber se comporta similar a MSE cuando el error es pequeño, pero al aumentar el error, comienza a penalizar menos.

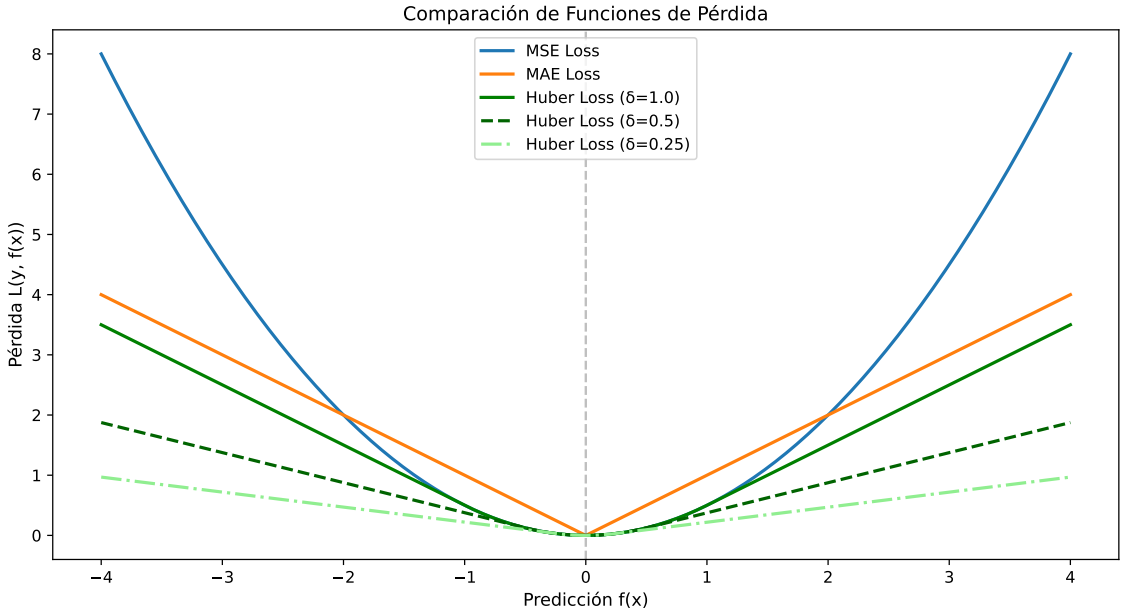

Figura 3.3: Comparación de funciones de pérdida: MSE, MAE y Huber.

3.2.3. Modelos base

Si bien el algoritmo de *Boosting* puede utilizar múltiples modelos base, en este trabajo se expondrán únicamente los modelos base de árboles de decisión. En particular, la literatura sugiere utilizar árboles de poca profundidad en este tipo de modelos. Siendo los *stumps* los árboles de profundidad 1 los más utilizados.

En la Figura 3.4 se puede ver el efecto de la profundidad de los árboles en el error del modelo donde queda claro como arboles con poca profundidad tienen errores más bajos que los árboles de profundidad mayor.

3.2.4. Regularización

Uno de los principales problemas de los modelos de aprendizaje automático es el sobreajuste. Es decir, que el modelo se ajuste demasiado a los datos de entrenamiento y por lo tanto no generalice bien a nuevos datos. Esto se da porque las funciones que se utilizan son muy flexibles y pueden llegar a ajustarse al conjunto de datos, por lo que existen determinadas técnicas para evitar esto. En este trabajo, se estudiarán tres técnicas: *subsampling*, *shrinkage* y *early stopping*.

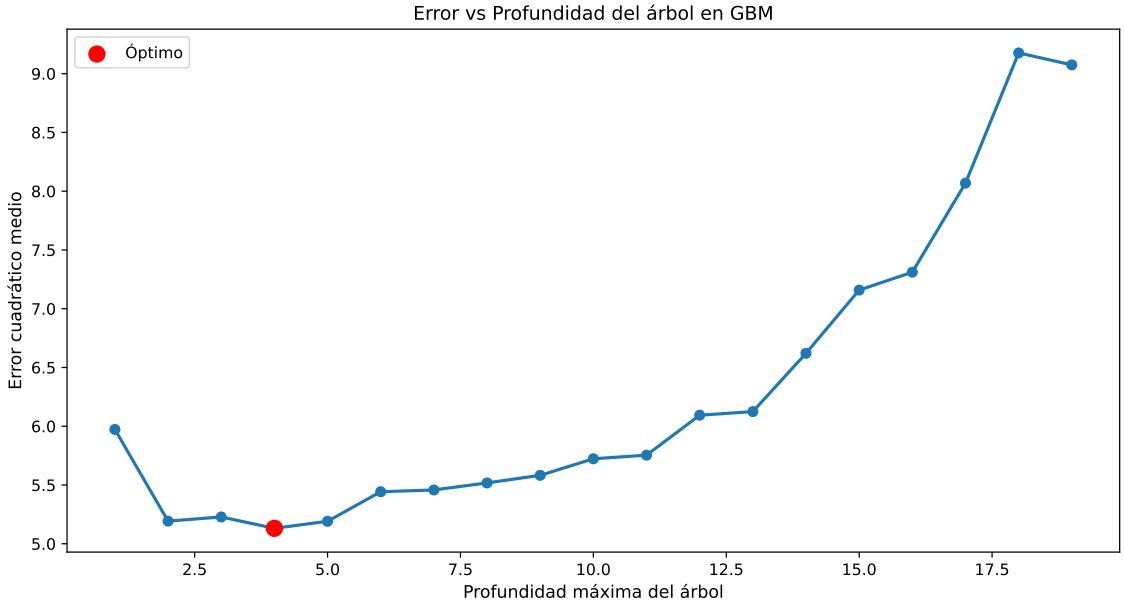

Figura 3.4: Error en función de la profundidad del árbol.

Subsampling

La idea intuitiva detrás del *subsampling* es introducir variabilidad en el entrenamiento del modelo. Esto se logra al considerar un subconjunto de las observaciones para cada iteración del algoritmo. El muestreo puede ser con o sin reemplazo. Cabe aclarar que para definir la proporción de la muestra que se utiliza, se utiliza el hyperparámetro *bag fraction* que toma valores entre 0 y 1. Por ejemplo, si se tiene una muestra de 1000 observaciones y se utiliza un *bag fraction* de 0.1, se utilizarán 100 observaciones para cada iteración.

Pese a que esta técnica puede ayudar a en casos donde los *datasets* sean muy grandes y el costo computacional sea alto, al solo considerar una parte de las observaciones el entrenamiento puede ser menos preciso por lo que se debe balancear.

Shrinkage

Shrinkage se utiliza para reducir el efecto de los árboles individuales en el modelo. Intuitivamente, es un hiperparámetro que permite regular cuánto se toman en cuenta los errores de los árboles anteriores para mejorar poco a poco las predicciones.

En definitiva, el shrinkage puede incluirse $\hat{f}_b(x)$ como:

$$\hat{f}_b(x) = \hat{f}_{b-1}(x) + \lambda\gamma_b h_b(x, \theta_b) \tag{3.24}$$

donde λ es el *shrinkage* que toma valores entre 0 y 1. Lógicamente, a medida que λ se acerca a 0, los incrementos de las predicciones son cada vez más pequeños y por lo tanto el modelo es menos sensible a los errores de los árboles anteriores, alcanzando así una mejor generalización. Además, al elegir un *shrinkage* pequeño, el modelo converge más lento, por lo cual se necesita un mayor número de iteraciones B para que el modelo llegue al mismo error.

En la Figura 3.5 se puede ver el efecto del shrinkage en el error del modelo tanto en la muestra de entrenamiento como en la muestra de testeo. Puede verse que en la muestra de entrenamiento el error disminuye más rápido a medida que aumenta

el shrinkage. Sin embargo, en la muestra de testeo puede verse como el *shrinkage* afecta tanto la velocidad de convergencia como la capacidad de generalización del modelo.

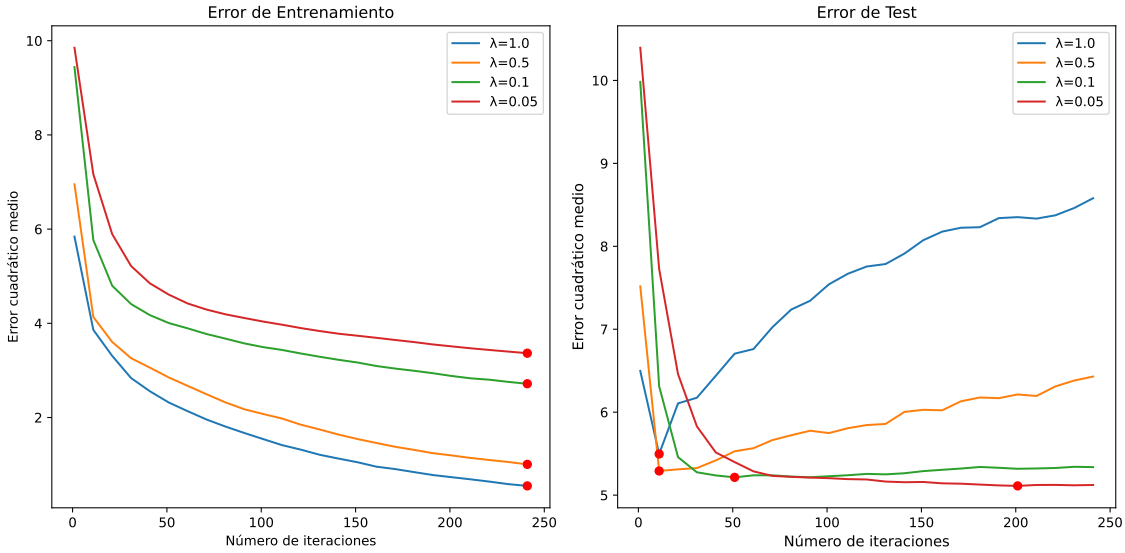

Figura 3.5: Comparación del error de testeo y entrenamiento en función de distintos valores de *shrinkage* y cantidad de árboles.

Early Stopping

Puede verse también en la Figura 3.5 que el modelo converge a un error mínimo. Sin embargo, el modelo puede sobreajustarse a los datos de entrenamiento y por lo tanto no generalizar bien a nuevos datos. Para evitar esto, se puede utilizar el *early stopping* que consiste en detener el entrenamiento cuando el error en la muestra de test deja de disminuir o lo hace de manera muy lenta.

De esta forma, se puede evitar el sobreajuste y reducir el tiempo de entrenamiento de estos modelos. Esto último es particularmente útil cuando se trabaja con muestras grandes.

4. Interpretabilidad

Tal como se mencionó anteriormente, uno de los inconvenientes de los ensambles de árboles es que al agrupar múltiples árboles el modelo en conjunto deja de ser interpretable. Esto puede no suponer un problema si el objetivo es predecir, pero si el objetivo es interpretar el modelo el hecho de trabajar con 'cajas negras' puede ser un problema.

Es por ello que se desarrollaron técnicas para extraer información de modelos de aprendizaje automático. Estas técnicas se aplican una vez entrenado el modelo y permiten entender cómo el mismo utiliza las variables. En este trabajo se estudiarán métodos para calcular la importancia de cada covariable y el efecto que tiene cada una de ellas sobre la variable dependiente.

4.1. Importancia de las variables

Con el acceso a información que se tiene actualmente, no es extraño querer incluir múltiples covariables en un mismo modelo. Sin embargo, entender cuáles de ellas realmente aportan al mismo no es trivial, por lo que se desarrollaron múltiples técnicas para evaluarlo. Esta sección presenta dos enfoques fundamentales: *Split Gain Feature Importance* y *Permutation Feature Importance*. Posteriormente, se analiza una limitación importante de estas técnicas (la inestabilidad de las importancias entre modelos con rendimiento similar) y se presenta *Model Class Reliance* como una solución a este problema.

4.1.1. *Split Gain Feature Importance*

Esta técnica fue propuesta por [Friedman, 2001] y asigna importancias en base a la reducción del error que cada variable genera al ser utilizada como criterio de división. Esta metodología es específica para árboles de decisión y se define formalmente para un solo árbol como:

$$\hat{I}_j^2(T) = \sum_{t=1}^{N-1} \hat{i}_t^2 1(v_t = j) \tag{4.1}$$

donde $\hat{I}_j^2(T)$ representa la importancia de la variable j en el árbol T, $\hat{i}_t^2$ es la mejora en el criterio de división del nodo t, y v_t indica la variable utilizada para dividir dicho nodo. La suma se realiza sobre todos los nodos no terminales.

Cuando se trabaja con ensambles de árboles, la importancia se generaliza promediando sobre todos los árboles del ensamble:

$$\hat{\mathbf{I}}_j^2 = \frac{1}{B}\sum_{m=1}^{B} \hat{I}_j^2(T_n) \tag{4.2}$$

Esta medida presenta limitaciones importantes: está basada en heurísticas sin fundamentos estadísticos sólidos, [Strobl et al., 2008] critica que otorga demasiada influencia a variables correlacionadas y favorece a variables categóricas con muchas categorías, dado que estas son propensas a generar más divisiones sin que esto implique una mayor contribución a la reducción del error.

4.1.2. *Permutation Feature Importance*

Una alternativa es *Permutation Feature Importance* (PFI), técnica desarrollada por [Breiman, 2001a] para *Random Forest* y posteriormente generalizada para cualquier modelo por [Fisher et al., 2019]. La intuición detrás de esta técnica es medir cómo cambia el error del modelo cuando se elimina la relación entre una variable y la respuesta mediante una permutación aleatoria. Las variables menos importantes serán aquellas cuya permutación no genere un gran mayor incremento en el error dado que si el modelo no utilizó la variable durante el entrenamiento, el hecho de permutar las observaciones de dicha variable no debería afectar el error. Por otro lado, se espera que para variables poco importantes los errores se mantengan similares con y sin la permutación.

Este proceso se repite K veces para cada variable con el fin de reducir la varianza introducida por la aleatoriedad de la permutación. El valor final de importancia se obtiene promediando los resultados de estas iteraciones. El número de repeticiones K suele definirse buscando un equilibrio entre la estabilidad de la métrica y el costo computacional, el cual depende directamente del tamaño del conjunto de datos y la velocidad de inferencia del modelo.

En el contexto de *Random Forest*, habiendo entrenado un modelo $\hat{f}$, para cada covariable $j \in \{1, 2, ..., p\}$ se construye un nuevo conjunto de datos permutando las observaciones de dicha variable mientras se mantienen las demás fijas. Luego se calcula el error *out-of-bag*:

$$E_{OOB}(X_j) = \frac{1}{n}\sum_{i=1}^{n} \left(y_i - \hat{f}_{OOB}(X_{i,j})\right)^2 \tag{4.3}$$

donde $\hat{f}_{OOB}(X_{i,j})$ representa la predicción agregada para la observación i, calculada utilizando únicamente aquellos árboles en los que i no formó parte de la muestra de entrenamiento, evaluada sobre el conjunto de datos con la variable j permutada.

La importancia se obtiene comparando este error con el del modelo original:

$$PFI(j) = E_{OOB}(X_j) - E_{OOB}(X) \;\; \text{ó} \;\; PFI(j) = \frac{E_{OOB}(X_j)}{E_{OOB}(X)} \tag{4.4}$$

Generalización: *Model Reliance*

La formulación original de PFI depende del error *out-of-bag*, lo que limita su aplicabilidad a modelos basados en *bagging*. [Fisher et al., 2019] proponen una generalización denominada *Model Reliance* (MR) que es aplicable a cualquier modelo.

Partiendo de un modelo entrenado $\hat{f}$, una variable de respuesta y, una función de pérdida L y una matriz de diseño X, para cada covariable j se permutan sus valores y se calcula la pérdida esperada:

$$L_{perm}(f) = \mathcal{E}[L(f, (Y, X_1^{perm}, X_2))] \tag{4.5}$$

donde X_1^{perm} representa la variable permutada y X_2 las demás variables que permanecen constantes. La importancia se define como el cociente entre la pérdida con permutación y la pérdida original:

$$MR^j(f) = \frac{L_{perm}^j(f)}{L(f)} \tag{4.6}$$

Para estimar L_{perm} en la práctica, los autores proponen dos alternativas. La primera realiza todas las permutaciones posibles:

$$\hat{e}_{perm}(F) = \frac{1}{n(n-1)} \sum_{i=1}^{n} \sum_{j \neq i} L\{f, y_j, X_{1[i,.]}, X_{2[i,.]}\} \tag{4.7}$$

Dado que esto puede ser computacionalmente intensivo, proponen como alternativa dividir la muestra a la mitad e intercambiar los valores de la variable de interés entre ambas mitades:

$$\begin{aligned} \hat{e}_{\text{divide}}(f) = \frac{1}{2\lfloor n/2 \rfloor} \sum_{i=1}^{\lfloor n/2 \rfloor} \Bigg[& L\Big(f, \Big(y_{[i]}, X_{1\left[i+\lfloor n/2 \rfloor\right]}, X_{2[i]}\Big)\Big) \\ & + L\Big(f, \Big(y_{i+\lfloor n/2 \rfloor}, X_{1[i]}, X_{2\left[i+\lfloor n/2 \rfloor\right]}\Big)\Big)\Bigg]. \end{aligned} \tag{4.8}$$

4.1.3. El problema de la inestabilidad: el *Rashomon Effect*

Las técnicas de importancia presentadas anteriormente comparten una limitación fundamental: la selección del modelo final se basa únicamente en una métrica de error, pero modelos con rendimiento predictivo similar pueden asignar importancias muy diferentes a las variables.

Para ilustrar este fenómeno, se entrenaron múltiples *Gradient Boosting Machines* con diferentes combinaciones de hiperparámetros sobre el conjunto de datos *Abalone* [Nash et al., 1994], donde el objetivo es predecir la edad del organismo en función de características físicas. Se evaluaron 324 combinaciones de hiperparámetros: cantidad de árboles de 100 a 900, tasas de aprendizaje de 0.01, 0.05 y 0.1, profundidad máxima de 2 a 5, y funciones de pérdida cuadrática, error absoluto y Huber. La selección se realizó mediante validación cruzada con $k = 5$.

La Tabla 4.1 presenta los 15 modelos con menor error. Se observa que múltiples configuraciones de hiperparámetros logran un rendimiento predictivo prácticamente idéntico.

Sin embargo, cuando se calculan las importancias mediante PFI para estos modelos, surge un resultado preocupante. La Figura 4.1 muestra cómo las importancias varían considerablemente entre modelos con rendimiento similar. En particular, se ve como una misma variable puede ocupar el cuarto lugar de importancia en un modelo y el séptimo en otro. Esta inconsistencia cuestiona la interpretabilidad de los resultados.

Cuadro 4.1: Resultados de los 15 modelos con menor error de predicción y sus hiperparámetros.

Modelo	RMSE	Shrinkage	Función de Pérdida	Profundidad máx	Árboles
RF1	2.1488	0.10	Huber	3	200
RF2	2.1498	0.05	Huber	4	200
RF3	2.1498	0.01	Huber	4	900
RF4	2.1506	0.10	Huber	4	100
RF5	2.1513	0.01	Pérdida cuadrática	4	600
RF6	2.1516	0.05	Pérdida cuadrática	4	200
RF7	2.1518	0.05	Huber	3	300
RF8	2.1518	0.01	Pérdida cuadrática	4	700
RF9	2.1519	0.05	Huber	3	400
RF10	2.1522	0.01	Pérdida cuadrática	4	800
RF11	2.1522	0.01	Huber	4	800
RF12	2.1526	0.05	Pérdida cuadrática	4	100
RF13	2.1530	0.01	Pérdida cuadrática	4	500
RF14	2.1531	0.10	Pérdida cuadrática	4	100
RF15	2.1537	0.01	Pérdida cuadrática	4	900

Este fenómeno se conoce como el *Rashomon Effect* [Breiman, 2001b], término inspirado en la película de Kurosawa donde un mismo evento es narrado desde perspectivas contradictorias. [Fisher et al., 2019] formalizan este concepto definiendo el ¨conjunto de Rashomon¨ como la colección de modelos con rendimiento predictivo similar pero estructuras internas diferentes. La existencia de múltiples explicaciones igualmente válidas representa un desafío fundamental: la importancia de una variable no es una propiedad intrínseca de los datos, sino que depende del modelo específico elegido.

4.1.4. *Model Class Reliance*: una solución al *Rashomon Effect*

Para abordar la inestabilidad de las importancias, [Fisher et al., 2019] proponen *Model Class Reliance* (MCR). En lugar de reportar la importancia de un único modelo, MCR calcula el rango de importancias posibles sobre todos los modelos con rendimiento similar al óptimo.

Primero se define formalmente el conjunto de Rashomon:

$$R(\epsilon) = \{f \in \mathcal{F} \mid e_{orig}(f) \leq e_{orig}(f_{ref}) + \epsilon\} \tag{4.9}$$

Este conjunto contiene todos los modelos de la clase $\mathcal{F}$ cuya pérdida sea como máximo ϵ mayor que la del modelo de referencia f_{ref} que minimiza el error.

Luego, el intervalo MCR se define como:

$$[MCR^{-}(\epsilon), MCR^{+}(\epsilon)] = [\min_{f \in R(\epsilon)} MR(f), \max_{f \in R(\epsilon)} MR(f)] \tag{4.10}$$

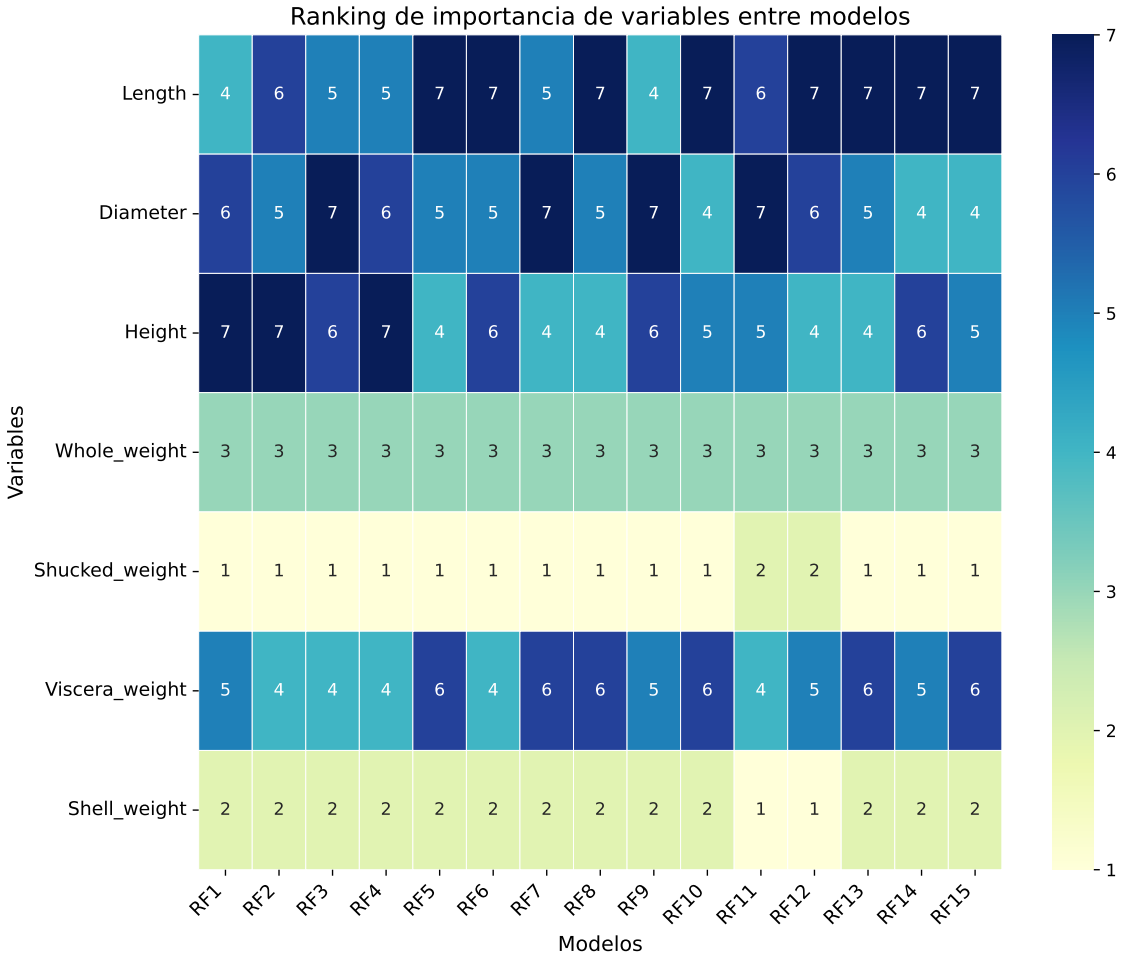

Figura 4.1: Mapa de calor del ranking de importancias para distintos modelos de *Gradient Boosting* con rendimiento similar.

Interpretación

El intervalo MCR proporciona información sobre la consistencia de la importancia de una variable. Si $MCR^{-}(\epsilon) > 1$, todos los modelos dentro del conjunto de Rashomon dependen de la variable, indicando que importante. Si $MCR^{+}(\epsilon) \approx 1$, ningún modelo depende fuertemente de ella.

La amplitud del intervalo también es informativa: un intervalo estrecho indica que la importancia es estable independientemente del modelo elegido, mientras que un intervalo amplio señala que la importancia depende fuertemente de la arquitectura particular. Por ejemplo, si para una variable j se tiene $MCR^{-}(\epsilon) \approx 1$ pero $MCR^{+}(\epsilon) \gg 1$, existen modelos con buena capacidad predictiva que no dependen de dicha variable, por lo que su importancia es cuestionable.

Estimación

En la práctica, MCR se estima mediante un estimador *plug-in*: se calcula $\hat{MR}(f)$ para todos los modelos que integren el conjunto de Rashomon empírico $\hat{R}(\epsilon)$, y se reportan los valores mínimo y máximo como límites del intervalo.

4.2. Efecto de las variables

Mientras que las técnicas de importancia de variables permiten identificar qué covariables son relevantes para el modelo, no proporcionan información sobre la forma de la relación entre cada variable y la respuesta. Para ello se desarrollaron técnicas de visualización que permiten aislar y representar el efecto de una o más covariables sobre la predicción del modelo.

Esta sección presenta tres técnicas complementarias: los *Partial Dependence Plots* (PDP), que estiman el efecto marginal promedio de una variable; los *Individual Conditional Expectation* (ICE), que permiten visualizar heterogeneidad en los efectos; y las variantes centradas y derivadas que facilitan la identificación de interacciones.

4.2.1. *Partial Dependence Plots* (PDP)

Los PDP fueron introducidos por [Friedman, 2001] como una técnica para visualizar el efecto marginal de una o más covariables sobre la predicción del modelo. La idea central es aislar el efecto de un subconjunto de variables X_S marginalizando sobre las demás variables X_C.

Formalmente, la dependencia parcial se define como:

$$\bar{f}(X_S) = \mathcal{E}_{X_C}[\hat{f}(x)] = \int \hat{f}(X_S, X_C)p(X_C)dX_C \tag{4.11}$$

donde $\hat{f}(x)$ es el modelo entrenado, X_S es el subconjunto de variables cuyo efecto se desea estimar, X_C representa las variables complementarias a X_S, $p(X_C)$ es su distribución marginal, y $\mathcal{E}_{X_C}[\cdot]$ denota la esperanza sobre dicha distribución.

En la práctica, la distribución $p(X_C)$ es desconocida y se aproxima mediante la distribución empírica de los datos de entrenamiento:

$$\bar{f}(X_S) \approx \frac{1}{n}\sum_{i=1}^{n} \hat{f}(X_S, x_C^i) \tag{4.12}$$

El algoritmo para calcular el PDP procede de la siguiente manera: para cada valor de interés x_S de la variable a analizar, se construyen n observaciones combinando x_S con los valores observados $x_C^{(i)}$ de las demás variables para cada observación i en la muestra. El modelo genera predicciones $\hat{f}(x_S, x_C^{(i)})$ para cada una de estas combinaciones, y el valor del PDP en x_S es el promedio de estas n predicciones. Este procedimiento se repite para una grilla de valores de x_S, generando la curva de dependencia parcial.

Esta metodología tiene limitaciones importantes. Por ejemplo, el supuesto implícito es que las variables X_S son independientes de X_C, lo cual raramente se cumple en la práctica. Además, cuando existen relaciones entre variables, el PDP puede generar predicciones para combinaciones de valores que no existen en los datos reales. Por ejemplo, si se analiza el efecto de la altura sobre el peso predicho, el PDP evaluará combinaciones donde una persona mide 2 metros pero pesa 50 kg, una combinación improbable que puede distorsionar la interpretación.

Para ilustrar los PDP se utilizó el mismo modelo de *Gradient Boosting Machine* entrenado anteriormente sobre el conjunto de datos *Abalone*. La Figura 4.2 muestra

el efecto del peso total sobre la edad predicha. Se observa una relación positiva: a mayor peso, mayor edad predicha, aunque la relación se aplana para pesos elevados.

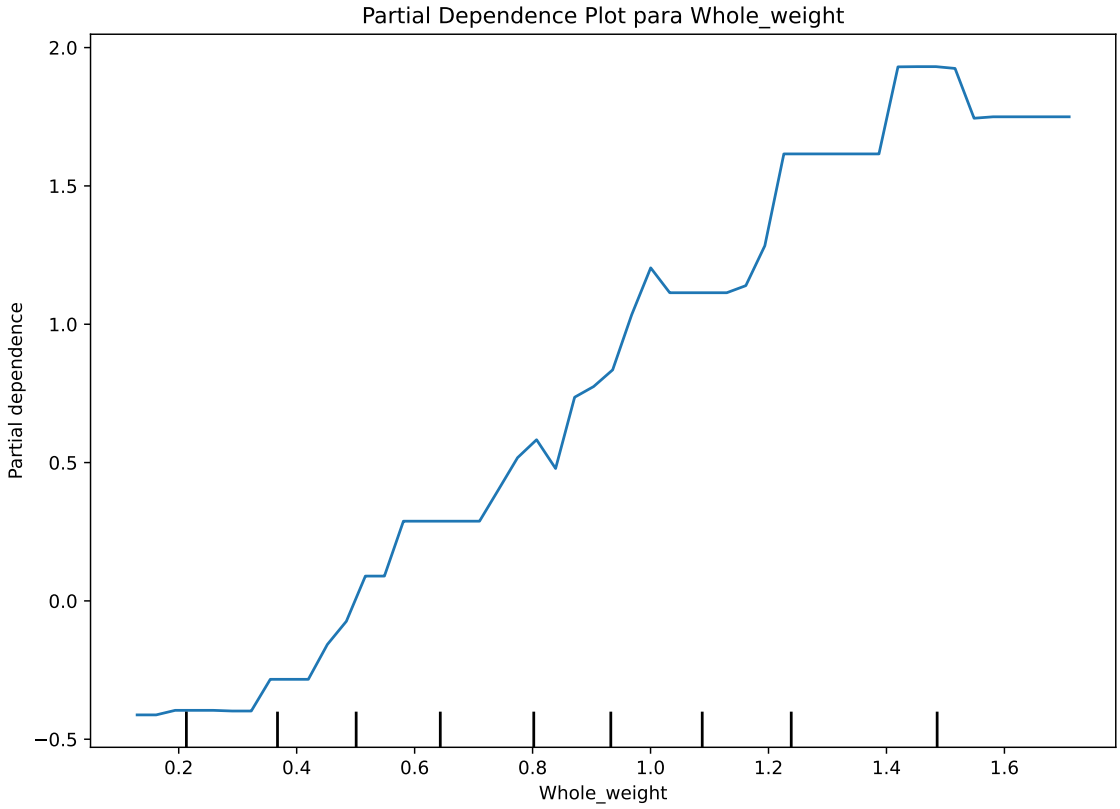

Figura 4.2: Gráfico de dependencia parcial para el peso total en la predicción de la edad del *Abalone*.

4.2.2. *Individual Conditional Expectation* (ICE)

Los PDP muestran el efecto promedio de una variable, pero este promedio puede ocultar heterogeneidad importante. [Goldstein et al., 2015] propusieron los gráficos ICE como una extensión de los PDP que permite visualizar el efecto de una variable para cada observación individual.

Formalmente, para cada observación i se define:

$$\hat{f}_i(x_S) = \hat{f}(x_S, x_C^{(i)}) \tag{4.13}$$

Es decir, en lugar de promediar sobre todas las observaciones, se mantienen fijos los valores $x_C^{(i)}$ de cada observación y se varía únicamente x_S. El resultado es una curva para cada observación en la muestra, lo que permite visualizar cómo varía el efecto de X_S dependiendo de los valores de las otras variables.

La relación entre PDP e ICE es directa: el PDP es simplemente el promedio de todas las curvas ICE:

$$\bar{f}(x_S) = \frac{1}{n} \sum_{i=1}^{n} \hat{f}_i(x_S) \tag{4.14}$$

Cuando las variables son independientes y no existen interacciones, todas las curvas ICE deberían ser paralelas y el PDP las resume adecuadamente. Sin embargo, cuando las curvas ICE divergen significativamente, esto indica la presencia de interacciones entre X_S y alguna de las variables en X_C. Por esta razón, se recomienda siempre visualizar las curvas ICE junto con el PDP para detectar posibles heterogeneidades.

La Figura 4.3 muestra las curvas ICE para la misma variable (peso total) analizada anteriormente. Se observa que, aunque la tendencia general es positiva, existe considerable variación entre observaciones. Esta dispersión sugiere que el efecto del peso sobre la edad predicha depende de los valores de otras características del organismo.

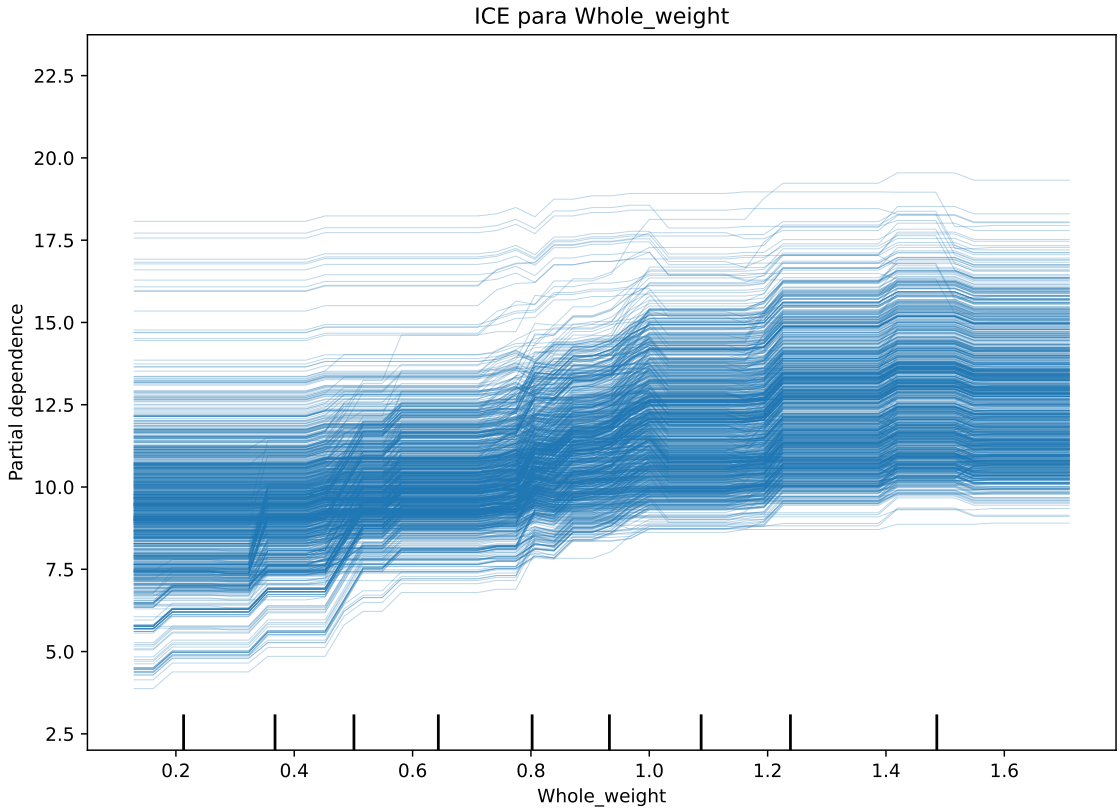

Figura 4.3: Gráfico de *Individual Conditional Expectation* (ICE) para el peso total en la predicción de la edad del *Abalone*.

Centered ICE (c-ICE)

Un problema práctico de los gráficos ICE es que las curvas tienen diferentes ordenadas al origen, lo que dificulta la comparación visual y la identificación de interacciones. Para abordar este problema, [Goldstein et al., 2015] proponen los gráficos ICE centrados, donde cada curva se centra en un punto de referencia común.

Formalmente, se elige un valor de referencia x^* (típicamente el mínimo o máximo de X_S) y se centra cada curva restando su valor en dicho punto:

$$\hat{f}^{(i)}_{cent}(x_S) = \hat{f}(x_S, x^{(i)}_C) - \hat{f}(x^*, x^{(i)}_C) \tag{4.15}$$

De esta manera, todas las curvas pasan por el origen en x^*. Si se elige el valor mínimo como referencia, las curvas muestran el efecto acumulado de incrementar X_S desde su valor mínimo. Esta representación facilita la identificación de interacciones.

La Figura 4.4 presenta el gráfico ICE centrado para el peso total, utilizando el valor mínimo como referencia. Se pueden observar diferencias en las pendientes de las curvas, lo que confirma la presencia de heterogeneidad en el efecto. La línea roja representa el PDP centrado, que resume el efecto promedio.

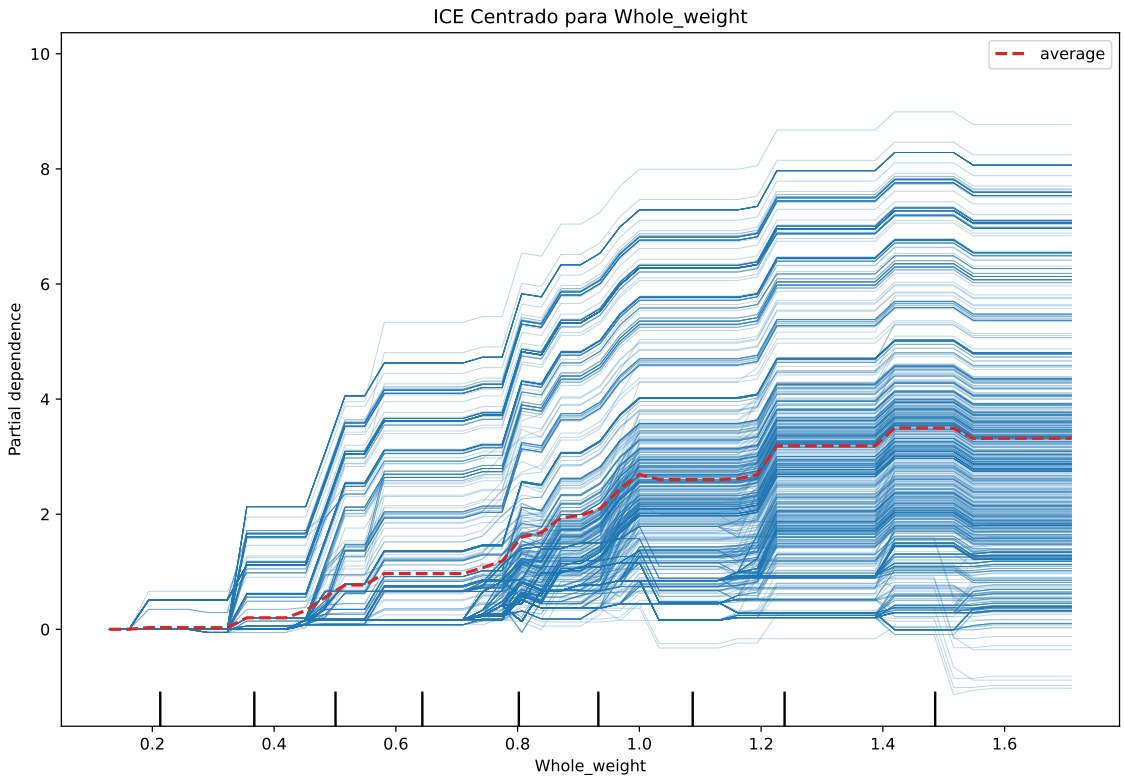

Figura 4.4: Gráfico de *Individual Conditional Expectation* (ICE) centrado para el peso total en la predicción de la edad del *Abalone*. La línea roja representa el PDP centrado.

Derivative ICE (d-ICE)

Como complemento a los gráficos centrados, [Goldstein et al., 2015] proponen los gráficos de la derivada de las curvas ICE. Este enfoque se basa en la observación de que, si no existen interacciones entre X_S y X_C, el modelo puede descomponerse aditivamente:

$$\hat{f}(x) = \hat{f}(x_S, x_C) = g(x_S) + h(x_C) \Rightarrow \frac{\partial \hat{f}(x)}{\partial x_S} = g'(x_S) \tag{4.16}$$

En este caso, la derivada de $\hat{f}$ con respecto a X_S depende únicamente de X_S y es idéntica para todas las observaciones. Por lo tanto, si las curvas derivadas coinciden, no existen interacciones; si difieren, la relación entre X_S y la predicción depende de los valores de otras variables. Esta herramienta es particularmente útil cuando las curvas ICE centradas son difíciles de interpretar visualmente debido a la cantidad de observaciones o a la complejidad de los patrones.

5. Aplicaciones

En esta sección se presentan tres casos de estudio para poner a prueba las técnicas descriptas: *Airfoil Self-Noise*, *Concrete Comprehensive Strength* y *Wine Quality*. El análisis de cada conjunto de datos comienza con una breve exploración y limpieza, seguida de una búsqueda de hiperparámetros (*grid search*) para minimizar el error de validación cruzada en el subconjunto de entrenamiento (80 % de los datos). Una vez optimizados, los modelos se evalúan en el conjunto de prueba restante para finalmente interpretar sus resultados mediante *Permutation Feature Importance* (PFI) y *Partial Dependence Plots* (PDP).

Se comparan cuatro tipos de modelos: regresión lineal, árboles de decisión, *Random Forest* y *Gradient Boosting Machines*. Su rendimiento se mide a través del Error Cuadrático Medio (RMSE), el Error Absoluto Medio (MAE) y el Coeficiente de Determinación (R^2).

La configuración de hiperparámetros explorada para los árboles de decisión incluye variaciones en la profundidad máxima (10, 20, 30) y los criterios de división (mínimo de observaciones para dividir: 2, 5, 10; mínimo por hoja: 1, 2, 4). Para los métodos de ensamble, *Random Forest* y *Gradient Boosting*, se ajusta el número de estimadores (de 100 a 2000), sumando en este último la tasa de aprendizaje o *learning rate* (0.01, 0.1, 0.2).

Por último, se analiza la estabilidad de la importancia de variables (PFI) entre los cinco mejores modelos de cada caso. La consistencia en estos resultados sugiere robustez en la interpretación; de lo contrario, podríamos estar ante la presencia de *Rashomon Sets*, donde modelos con capacidades predictivas similares ofrecen explicaciones contradictorias del fenómeno. El objetivo no es necesariamente obtener el modelo perfecto, sino ilustrar la aplicación práctica algunas de las metodologías de interpretabilidad discutidas.

5.1. *Airfoil Self-Noise*

El primer conjunto de datos proviene de la NASA y describe pruebas aerodinámicas y acústicas realizadas en un túnel de viento sobre diferentes secciones de palas [Brooks et al., 1989]. El objetivo es predecir la presión sonora (*scaled-sound-pressure*) a partir de características físicas y operativas. La muestra consta de 1503 observaciones completas (sin valores faltantes) que incluyen perfiles NACA 0012 de distintos tamaños sometidos a diversas velocidades y ángulos de ataque. Las variables disponibles se detallan en la Tabla 5.1.

El análisis de correlación inicial (Figura 5.1) reveló una fuerte asociación entre el espesor de desplazamiento (*suction-side-displacement-thickness*) y el ángulo de ataque (*attack-angle*). Dado que este último también presentaba una correlación

Cuadro 5.1: Variables del conjunto de datos *Airfoil Self-Noise* y su significado.

Variable	**Significado**
frequency	Frecuencia
attack-angle	Ángulo de ataque en grados
chord-length	Longitud de cuerda en metros
free-stream-velocity	Velocidad de flujo libre en metros por segundo
suction-side-displacement-thickness	Espesor de desplazamiento del lado de succión en metros
scaled-sound-pressure	Presión sonora en decibelios (var dep)

considerable con la longitud de cuerda (*chord-length*) pero menor influencia lineal sobre la variable objetivo, se optó por excluirlo del modelado para reducir la multicolinealidad.

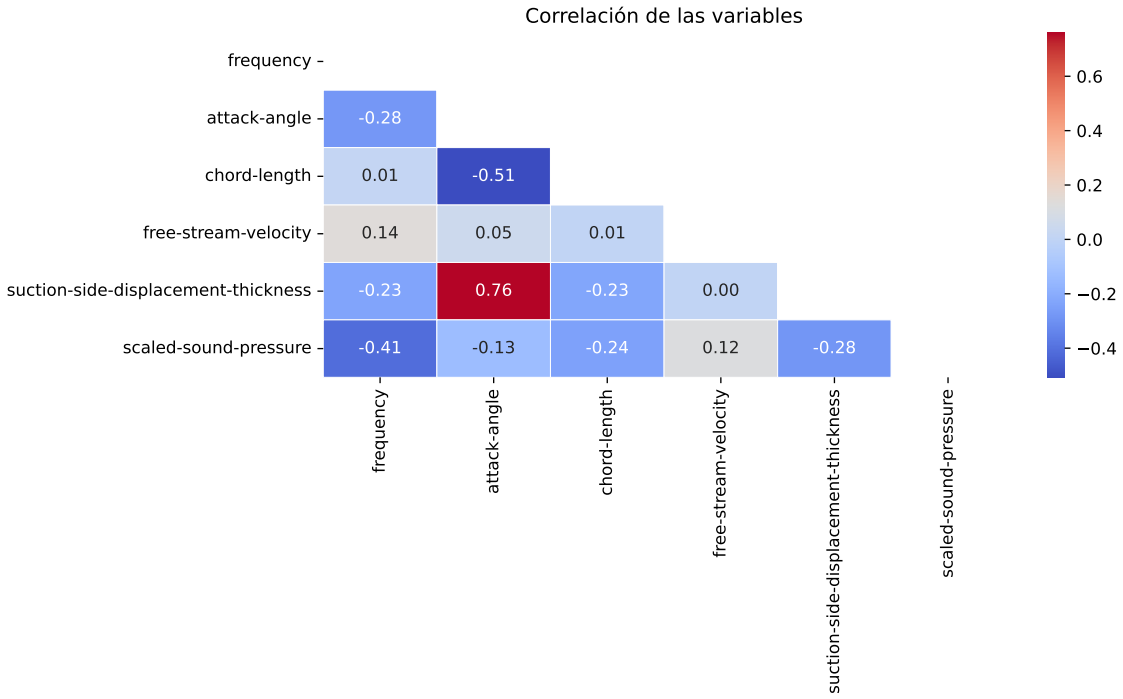

Figura 5.1: Correlación de las variables del conjunto de datos *Airfoil*.

Tras el entrenamiento, los resultados en el conjunto de prueba (Tabla 5.2) muestran que *Gradient Boosting* supera significativamente a los demás modelos en todas las métricas evaluadas. En contraste, la regresión lineal presenta el peor ajuste, un patrón que, como se verá más adelante, se repite en los tres casos de estudio, lo cual es esperable dada la complejidad y posibles no linealidades de los fenómenos físicos estudiados.

Para comprender qué factores influencian las predicciones del modelo de *Gradient Boosting*, se recurre a la importancia de variables por permutación (PFI). La Figura 5.2 señala a la frecuencia y el espesor de desplazamiento como los predictores dominantes, seguidos por la longitud de cuerda y la velocidad de flujo libre. Es relevante notar que los cinco modelos con mejor desempeño coincidieron en este orden de importancia, otorgando robustez a esta conclusión (Tabla A.2).

Para profundizar en la relación entre las variables más importantes y la presión

Cuadro 5.2: Resultados del mejor modelo de cada tipo evaluados en el conjunto de testeo para *Airfoil*.

Modelo	RMSE	MAE	R^2
Regresión Lineal	4.9766	3.9174	0.5056
Árbol de decisión	2.2708	1.7131	0.8971
Random Forest	1.8258	1.3177	0.9335
Gradient Boosting	1.4761	1.0337	0.9565

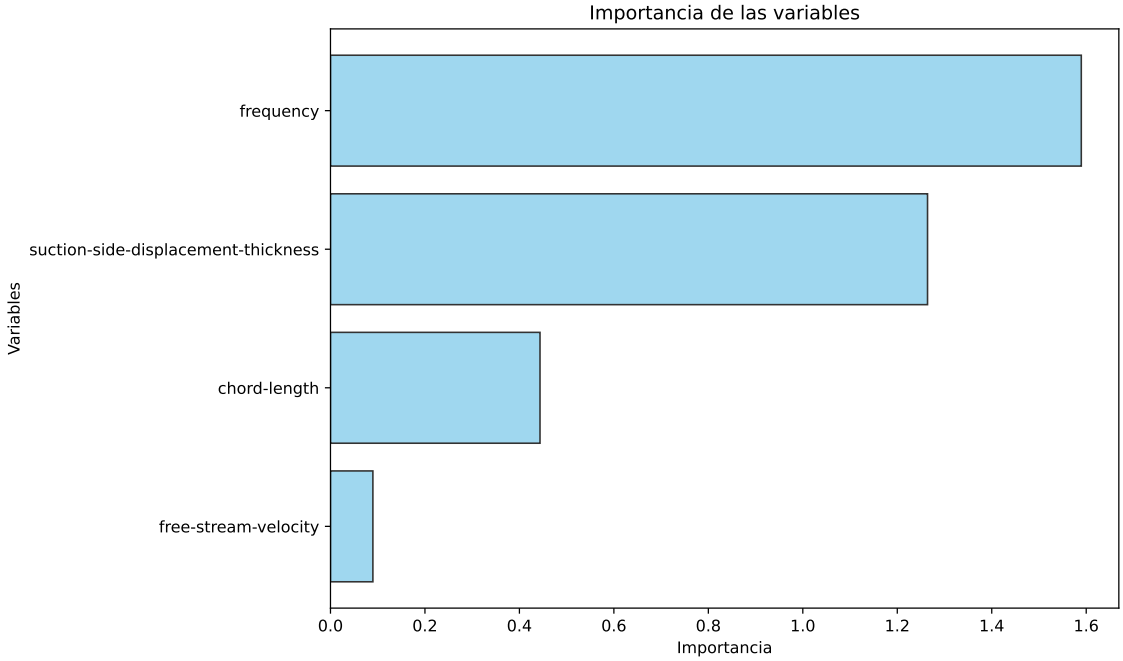

Figura 5.2: Importancia de variables (PFI) para el modelo de *Gradient Boosting* en el conjunto *Airfoil*.

sonora, se utilizan los gráficos de dependencia parcial (Figura 5.3). En el caso de la frecuencia, se observa una relación inversa clara: los niveles de presión sonora son máximos (130–140 dB) en el rango de bajas frecuencias (0–2500 Hz) y descienden progresivamente hasta los 100–110 dB cerca de los 20000 Hz. El análisis centrado corrobora este efecto, mostrando variaciones de hasta ±20 dB respecto a la media. No obstante, la escasez de observaciones por encima de los 7500 Hz sugiere interpretar con cautela las predicciones en ese extremo del espectro.

Por su parte, el espesor de desplazamiento muestra un comportamiento no lineal más complejo. Aunque la presión sonora fluctúa principalmente entre 120 y 130 dB, la curva de respuesta presenta múltiples discontinuidades y una leve tendencia descendente general. Al igual que con la frecuencia, la distribución de los datos es asimétrica, concentrándose la gran mayoría de las observaciones por debajo de 0.015 metros.

En síntesis, el análisis de este conjunto de datos demuestra la superioridad predictiva de los métodos de ensamble y permite identificar claramente a la frecuencia y el espesor de desplazamiento como los factores acústicos determinantes.

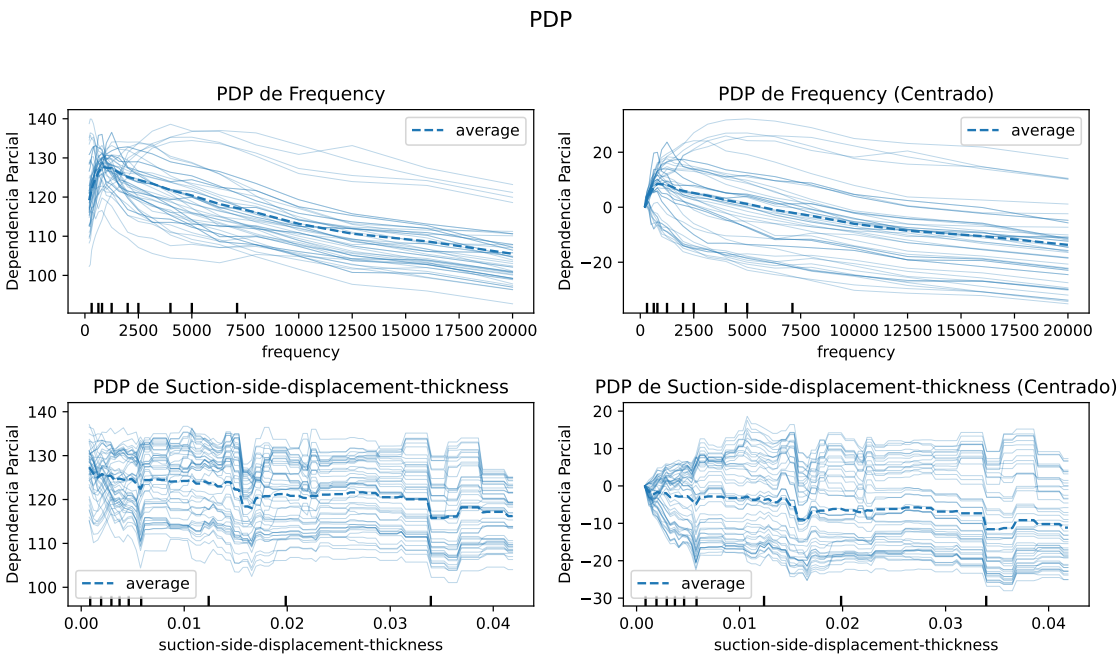

Figura 5.3: Gráficos de Dependencia Parcial (PDP) para *frequency* y *suction-side-displacement-thickness* en el modelo de *Gradient Boosting* (*Airfoil*).

5.2. *Concrete Compressive Strength*

El segundo caso de estudio utiliza el conjunto de datos *Concrete Compressive Strength* [Yeh, 1998]. Este consta de 1030 observaciones sobre la composición del concreto y su edad, con el fin de determinar su resistencia a la compresión. En la Tabla 5.3 se describen las 9 variables involucradas:

Cuadro 5.3: Variables del conjunto de datos *Concrete Compressive Strength* y su significado.

Variable	**Significado**
Cement	Cantidad de cemento en la mezcla (kg/m^3)
Blast Furnace Slag	Cantidad de escoria de alto horno (kg/m^3)
Fly Ash	Cantidad de cenizas volantes (kg/m^3)
Water	Cantidad de agua en la mezcla (kg/m^3)
Superplasticizer	Cantidad de superplastificante (kg/m^3)
Coarse Aggregate	Cantidad de agregado grueso (kg/m^3)
Fine Aggregate	Cantidad de agregado fino (kg/m^3)
Age	Edad del concreto en días
Concrete Comprehensive Strength	Resistencia a la compresión del concreto (var dep)

A diferencia del caso anterior, el análisis de correlación (Figura 5.4) no mostró coeficientes superiores a 0.8 en valor absoluto, por lo que se decidió conservar todas las variables para el modelado.

Al evaluar el rendimiento predictivo en el conjunto de prueba (Tabla 5.4), *Gradient Boosting* emerge nuevamente como el modelo con mejor desempeño, consolidando la tendencia observada previamente sobre la eficacia de los métodos de

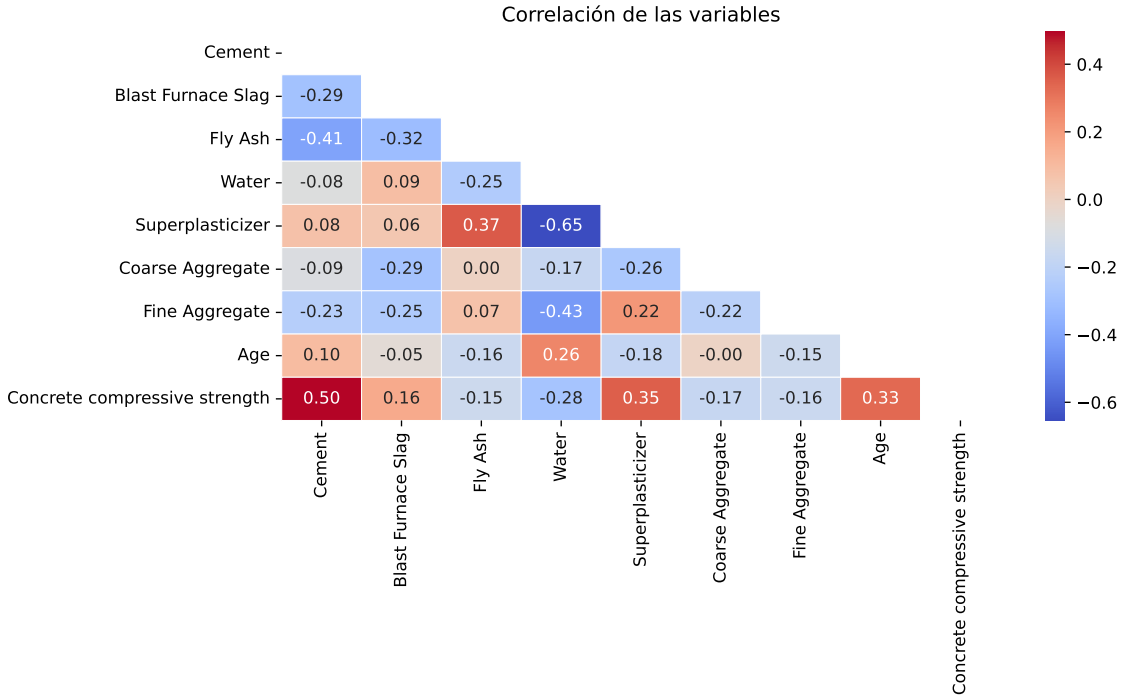

Figura 5.4: Matriz de correlación entre las variables del conjunto de datos *Concrete Compressive Strength*.

ensamble en estos contextos.

Cuadro 5.4: Resultados de los modelos en el conjunto de testeo para *Concrete Compressive Strength*.

Modelo	RMSE	MAE	R^2
Regresión Lineal	9.7965	7.7456	0.6276
Árbol de decisión	7.0675	4.8364	0.8062
Random Forest	5.5196	3.7957	0.8818
Gradient Boosting	4.3500	2.8629	0.9266

En cuanto a la interpretabilidad, el análisis de importancia de variables (Figura 5.5) identifica a la Edad y la Cantidad de Cemento como los factores más influyentes en la resistencia del concreto. Esta conclusión es consistente entre los cinco mejores modelos ajustados (Tabla A.4).

Los gráficos de dependencia parcial (Figura 5.6) permiten examinar en detalle el efecto de estas variables clave. Para la Edad, se aprecia un crecimiento pronunciado de la resistencia durante los primeros 50 días, tras lo cual la curva se estabiliza, indicando ganancias marginales más modestas con el paso del tiempo. Por su parte, la relación con el Cemento presenta un comportamiento escalonado con incrementos notables en umbrales específicos (250, 300 y 350 kg/m^3). Un hallazgo interesante de los gráficos ICE es que las curvas individuales son paralelas, lo que sugiere que el efecto de estas variables es aditivo y relativamente independiente de las interacciones con otros componentes de la mezcla.

En resumen, este análisis confirma que la maduración del concreto (edad) y su contenido de cemento son los determinantes críticos de su resistencia, siendo

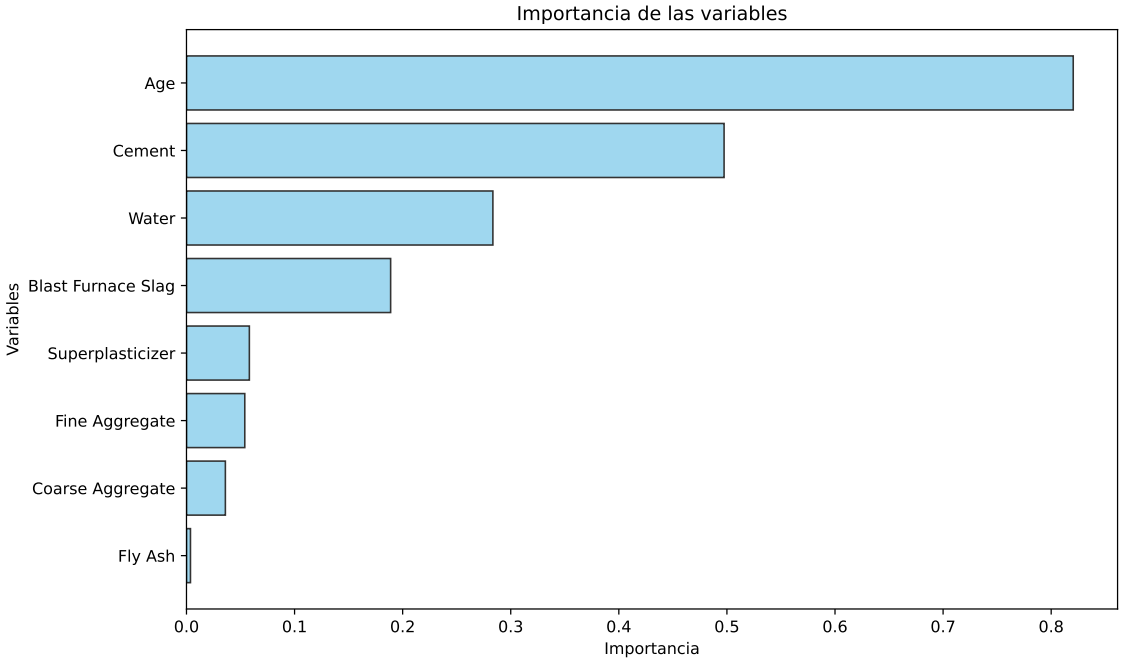

Figura 5.5: Importancia de las variables (PFI) del modelo *Gradient Boosting* para *Concrete Compressive Strength*.

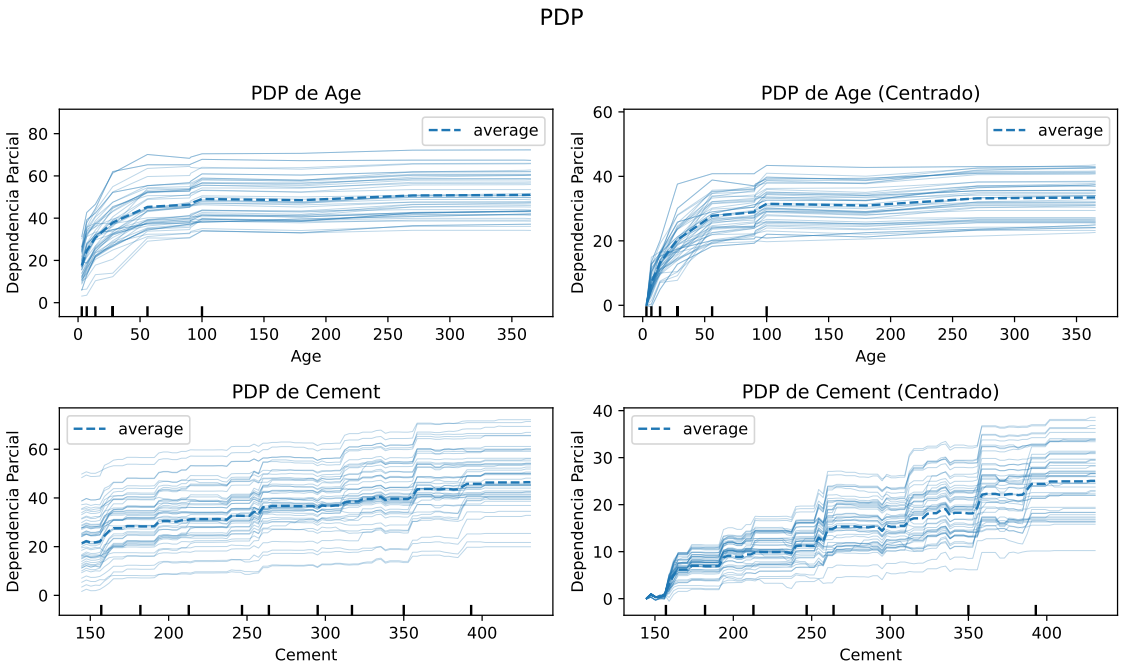

Figura 5.6: Partial dependence plots (PDPs) e individual conditional expectation (ICE) para edad y cemento en el conjunto de datos *Concrete Compressive Strength*.

Gradient Boosting el algoritmo más capaz de capturar estas relaciones.

5.3. *Wine Quality*

Finalmente, se analiza el conjunto de datos *Wine Quality* [Aeberhard and Forina, 1992], que contiene 4898 muestras de vinos blancos de una región específica de Italia. Se dispone de 11 variables fisicoquímicas (Tabla 5.5) para predecir la calidad sensorial del vino, medida en una escala discreta de 0 a 10.

Cuadro 5.5: Variables del conjunto de datos *Wine Quality* y su significado.

Variable	**Significado**
fixed_acidity	Acidez fija en el vino (principalmente ácido tartárico)
volatile_acidity	Acidez volátil en el vino (principalmente ácido acético)
citric_acid	Ácido cítrico presente en el vino
residual_sugar	Cantidad de azúcar residual en el vino
chlorides	Contenido de cloruros (sal) en el vino
free_sulfur_dioxide	Dióxido de azufre libre en el vino
total_sulfur_dioxide	Dióxido de azufre total en el vino (libre + combinado)
density	Densidad del vino
pH	Nivel de acidez/alcalinidad del vino (escala pH)
sulphates	Contenido de sulfatos en el vino
alcohol	Contenido de alcohol en el vino (% por volumen)
quality	Calidad del vino, puntuación entre 0 y 10 (var. dep.)

Antes del modelado, se detectó una alta colinealidad entre el dióxido de azufre total y el libre (Figura 5.7), procediéndose a eliminar este último.

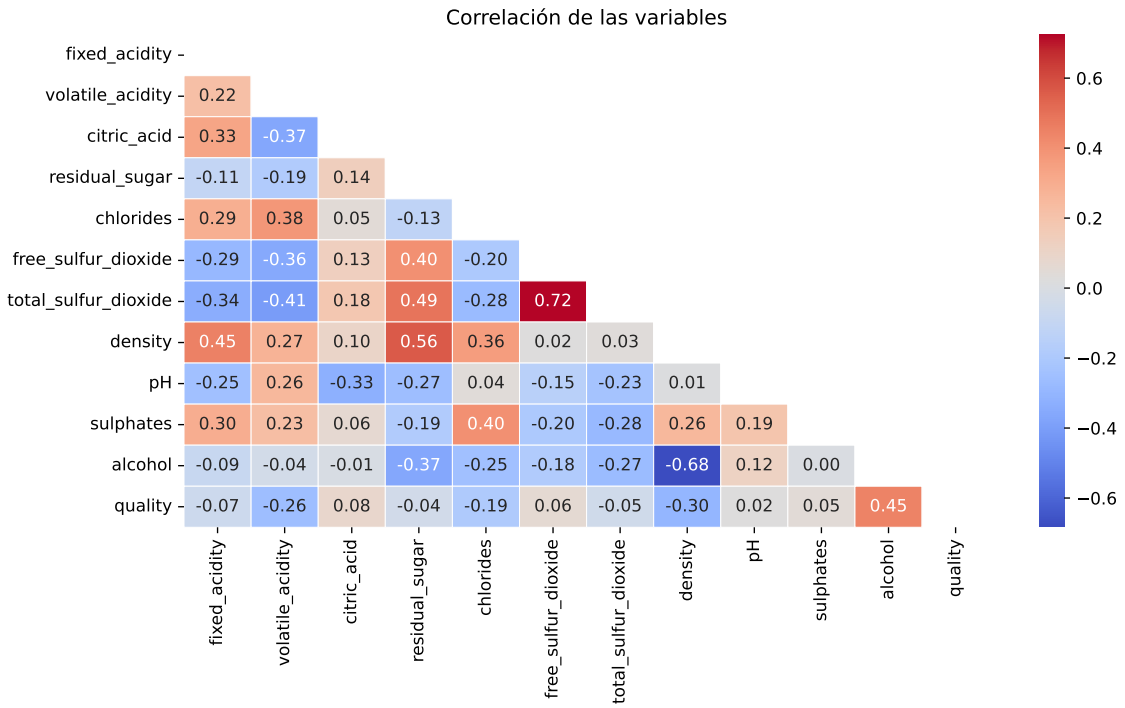

Figura 5.7: Matriz de correlación para las variables del conjunto de datos *Wine Quality*.

Los resultados en el conjunto de prueba (Tabla 5.6) presentan un cambio notable respecto a los casos anteriores: *Random Forest* obtiene los mejores resultados, superando incluso a *Gradient Boosting*. Aunque los valores de R^2 son en general más bajos que en los otros conjuntos de datos.

Cuadro 5.6: Resultados de los modelos en el conjunto de testeo para *Wine*.

Modelo	RMSE	MAE	R^2
Regresión Lineal	0.7378	0.5684	0.2630
Árbol de decisión	0.7697	0.5869	0.1979
Random Forest	0.6103	0.4379	0.4957
Gradient Boosting	0.6483	0.4818	0.4308

La importancia de variables (Figura 5.8) destaca rotundamente al contenido de alcohol como el principal predictor de la calidad, seguido por la acidez volátil. Esta jerarquía se mantiene estable a través de los diversos modelos ajustados (Tabla A.6).

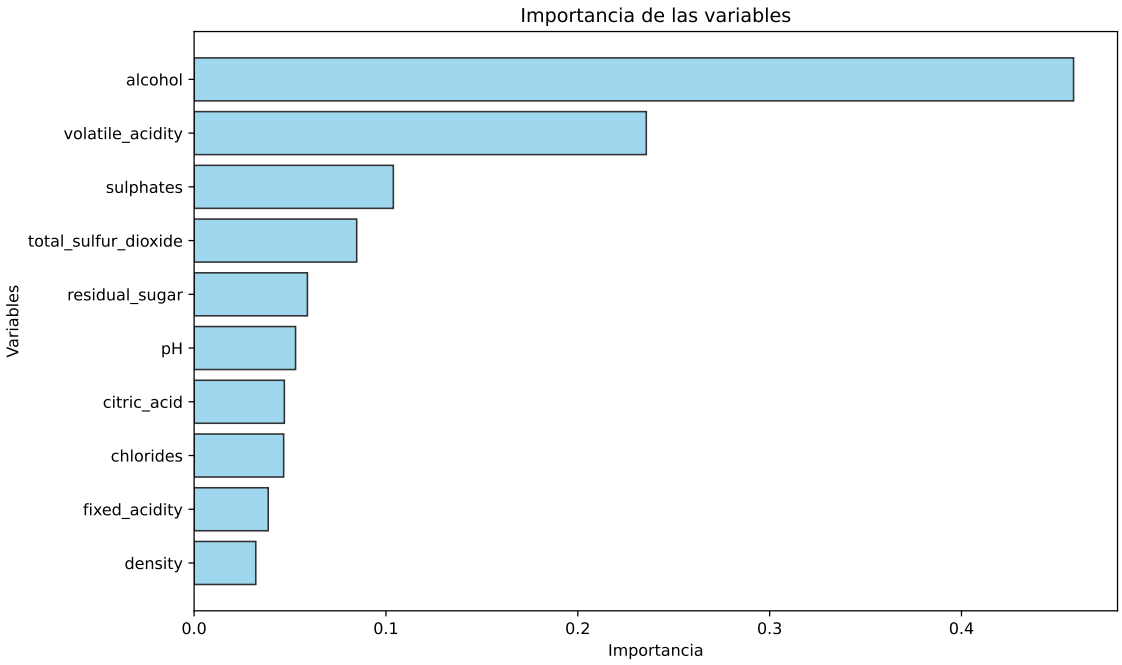

Figura 5.8: Importancia de variables (PFI) en el modelo de *Random Forest* para *Wine Quality*.

El análisis de dependencia parcial (Figura 5.9) muestra el efecto de estas variables. Para el alcohol, la relación con la calidad es positiva y no lineal, con un impacto mayor en el rango medio (11 %–13 %). Por el contrario, la acidez volátil penaliza la calidad de forma casi lineal: a medida que aumenta su concentración, la puntuación predicha disminuye.

En conclusión, la aplicación de estas técnicas en tres dominios distintos demuestra su versatilidad. Más allá de la precisión predictiva, donde los ensambles dominaron, herramientas como PFI y PDP permitieron interpretar el efecto de las variables en los modelos más complejos con mayor capacidad predictiva.

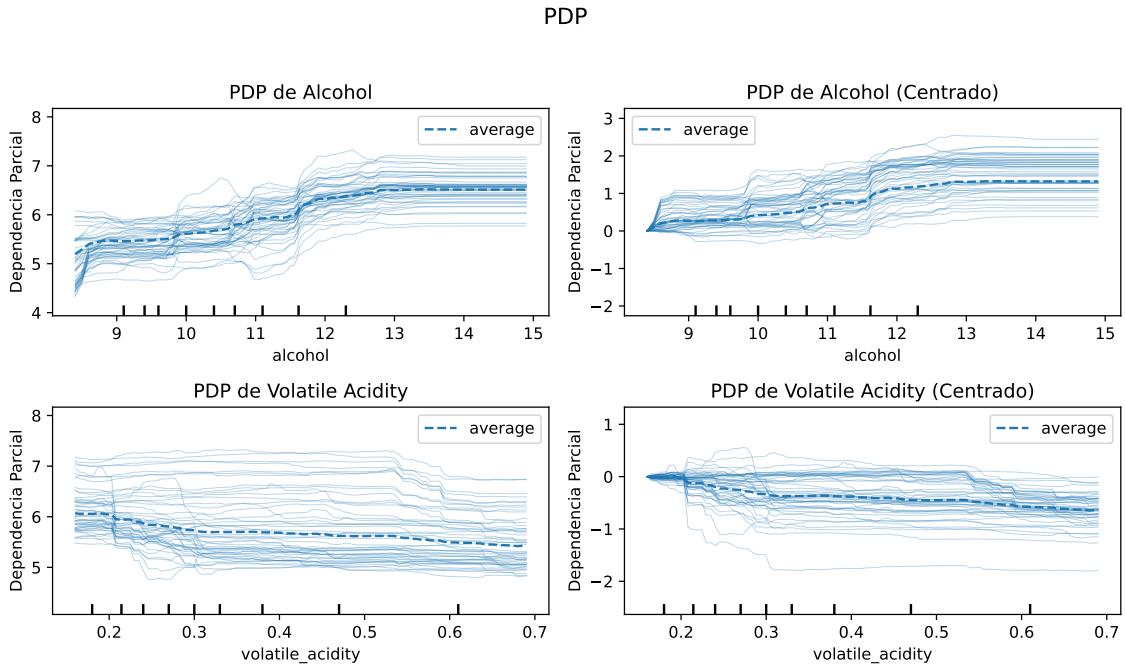

Figura 5.9: Gráficos de Dependencia Parcial (PDP) e ICE para alcohol y acidez volátil en el modelo de *Random Forest* (*Wine Quality*).

6. Conclusiones

Este trabajo se propuso abordar el compromiso o *trade-off* entre la capacidad predictiva y la capacidad explicativa. Se partió de la premisa de que, tradicionalmente, se debía elegir entre modelos simples e interpretables, como la regresión lineal, o modelos complejos de alta capacidad predictiva pero difícil interpretación, conocidos como cajas negras o *black boxes*.

A lo largo del desarrollo, se mostró que los ensambles de árboles, específicamente *Random Forest* y *Gradient Boosting Machines*, representan una buena solución para problemas de predicción, superando consistentemente a los modelos base y a las regresiones lineales en los casos de estudio analizados. Sin embargo, su complejidad planteaba el desafío de la interpretabilidad.

Para superar esta barrera, se estudiaron y aplicaron técnicas de aprendizaje automático interpretable. Se vio que la *permutation feature importance* permite identificar qué covariables son fundamentales para el modelo, mientras que los *partial dependence plots* e *individual conditional expectation* permiten visualizar cómo se relacionan estas variables con la variable dependiente. Además, se abordó el problema de la estabilidad y los *Rashomon Sets*, destacando que la interpretación debe ser cautelosa ante la existencia de múltiples modelos con rendimiento similar pero interpretaciones diferentes.

Es fundamental concluir este trabajo remarcando ciertas limitaciones. En primer lugar, la interpretación del modelo no es causal. Las técnicas presentadas solo describen cómo el modelo utiliza las variables para predecir. Si bien se discutió teóricamente, siguiendo a [Zhao and Hastie, 2021], que supuestos como la ausencia de *confounders* y la correcta especificación del DAG estas interpretaciones pueden tener una lectura causal, en la práctica, y especialmente con datos observacionales, estos supuestos raramente se pueden verificar en su totalidad. Por lo tanto, las conclusiones extraídas de estas herramientas deben entenderse principalmente en términos de asociación. Para realizar inferencia causal rigurosa, sería necesario complementar estas técnicas con marcos de trabajo específicos, como el *double machine learning* o el diseño experimental adecuado, que exceden el alcance de este estudio.

Otra limitación importante es la falta de una teoría de inferencia formal robusta similar a la que existe para la regresión lineal o la estadística clásica. Mientras que en los modelos lineales tradicionales se cuenta con intervalos de confianza, pruebas de hipótesis y propiedades asintóticas conocidas para los estimadores, en los modelos de aprendizaje automático estas garantías teóricas son mucho más difíciles de establecer. Si bien existen aproximaciones basadas en remuestreo, como el *bootstrapping*, para estimar la varianza y construir intervalos de confianza, estas técnicas suelen carecer del mismo rigor matemático y formalismo que caracteriza a la inferencia estadística clásica. Esto implica que las conclusiones sobre la importancia de una variable o los efectos deben tomarse como direccionales. Sin embargo, es relevante el

desarrollo de *Conformal Prediction* [Vovk et al., 2005]. Este enfoque permite pasar de predicciones puntuales a intervalos de predicción válidos en muestras finitas. De este modo, se dota a los modelos de caja negra de garantías estadísticas, un avance que ha cobrado gran interés reciente [Angelopoulos and Bates, 2023].

En definitiva, este trabajo evidencia que el aprendizaje automático interpretable es una herramienta poderosa para dotar de transparencia a los modelos predictivos, permitiendo a los usuarios confiar en sus decisiones, aunque siempre manteniendo una postura crítica respecto a sus limitaciones.

A. Apéndice

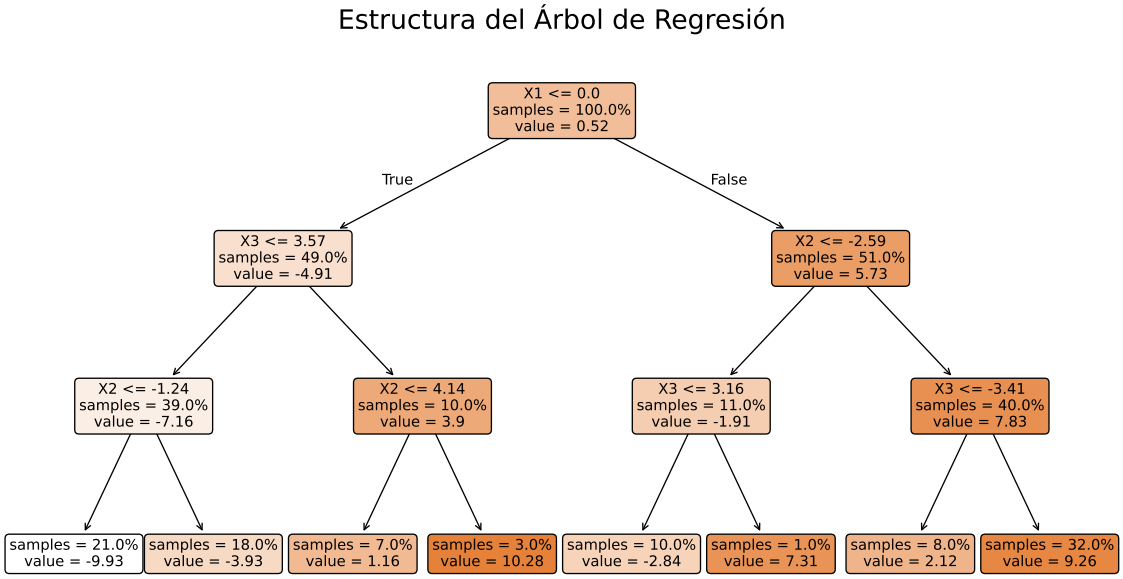

Figura A.1: Estructura del árbol de regresión sin valores atípicos.

Cuadro A.1: Metricas de los 5 modelos con menor error en la muestra de validación cruzada - Airfoil Self Noise

Rank	*Learning Rate*	*N Estimators*	RMSE	MAE	R^2
1	0.2	1900	1.4761	1.0337	0.9565
2	0.2	1800	1.4838	1.0420	0.9561
3	0.2	1700	1.4943	1.0508	0.9554
4	0.2	1600	1.5117	1.0663	0.9544
5	0.2	1500	1.5173	1.0773	0.9540

Cuadro A.2: Comparación de importancia de características para los 5 modelos con menor error en la muestra de validación cruzada - Airfoil Self Noise

Feature	Top 1	Top 2	Top 3	Top 4	Top 5
suction-side-displacement-thickness	0.4134	0.4133	0.4133	0.4131	0.4129
frequency	0.4084	0.4084	0.4085	0.4086	0.4087
chord-length	0.1385	0.1385	0.1386	0.1386	0.1386
free-stream-velocity	0.0397	0.0397	0.0397	0.0397	0.0397

Cuadro A.3: Metricas de los 5 modelos con menor error en la muestra de validación cruzada - Concrete Strength

Rank	*Learning Rate*	*N Estimators*	RMSE	MAE	R^2
1	0.1	800	4.3500	2.8629	0.9266
2	0.1	700	4.3934	2.9138	0.9251
3	0.1	900	4.3479	2.8536	0.9266
4	0.1	1000	4.3259	2.8221	0.9274
5	0.1	600	4.4356	2.9745	0.9236

Cuadro A.4: Comparación de importancia de características para los 5 modelos con menor error en la muestra de validación cruzada - Concrete Strength

Feature	Top 1	Top 2	Top 3	Top 4	Top 5
Age	0.3578	0.3578	0.3578	0.3579	0.3579
Cement	0.3021	0.3023	0.3020	0.3018	0.3024
Water	0.1139	0.1138	0.1138	0.1138	0.1139
Blast Furnace Slag	0.0811	0.0812	0.0811	0.0811	0.0812
Superplasticizer	0.0747	0.0747	0.0747	0.0747	0.0746
Fine Aggregate	0.0369	0.0368	0.0369	0.0369	0.0367
Coarse Aggregate	0.0189	0.0188	0.0190	0.0191	0.0187
Fly Ash	0.0146	0.0146	0.0146	0.0147	0.0146

Cuadro A.5: Metricas de los 5 modelos con menor error en la muestra de validación cruzada - Wine Quality

Rank	*Learning Rate*	*N Estimators*	RMSE	MAE	R^2
1	0.2	1600	0.6103	0.4379	0.4957
2	0.2	1300	0.6102	0.4379	0.4959
3	0.2	1400	0.6100	0.4378	0.4961
4	0.2	1900	0.6102	0.4378	0.4958
5	0.2	1800	0.6103	0.4379	0.4957

Cuadro A.6: Comparación de importancia de características para los 5 modelos con menor error en la muestra de validación cruzada - Wine Quality

Feature	Top 1	Top 2	Top 3	Top 4	Top 5
alcohol	0.2602	0.2602	0.2603	0.2601	0.2601
volatile_acidity	0.1397	0.1395	0.1395	0.1396	0.1396
total_sulfur_dioxide	0.0885	0.0885	0.0885	0.0887	0.0886
sulphates	0.0832	0.0833	0.0833	0.0831	0.0831
pH	0.0807	0.0806	0.0806	0.0808	0.0807
residual_sugar	0.0785	0.0787	0.0785	0.0786	0.0786
chlorides	0.0712	0.0710	0.0712	0.0712	0.0712
citric_acid	0.0681	0.0683	0.0682	0.0680	0.0681
density	0.0656	0.0656	0.0656	0.0655	0.0655
fixed_acidity	0.0644	0.0643	0.0643	0.0644	0.0643

Bibliografía

[Aeberhard and Forina, 1992] Aeberhard, S. and Forina, M. (1992). Wine. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5PC7J.

[Angelopoulos and Bates, 2023] Angelopoulos, A. and Bates, S. (2023). *Conformal Prediction: A Gentle Introduction.*

[Breiman, 1996] Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.

[Breiman, 2001a] Breiman, L. (2001a). Random forests. *Machine Learning*, 45(1):5–32.

[Breiman, 2001b] Breiman, L. (2001b). Statistical Modeling: The Two Cultures (with comments and a rejoinder by the author). *Statistical Science*, 16(3):199 – 231.

[Breiman et al., 1984] Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees.* Routledge.

[Brooks et al., 1989] Brooks, T., Pope, D., and Marcolini, M. (1989). Airfoil Self-Noise. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5VW2C.

[Fisher et al., 2019] Fisher, A., Rudin, C., and Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of machine learning research : JMLR*, 20.

[Friedman, 2001] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189 – 1232.

[Goldstein et al., 2015] Goldstein, A., Kapelner, A., Bleich, J., and Pitkin, E. (2015). Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1):44–65.

[James et al., 2023] James, G., Witten, D., Hastie, T., and Tibshirani, R. (2023). *An Introduction to Statistical Learning: with Applications in R.* Springer.

[Nash et al., 1994] Nash, W., Sellers, T., Talbot, S., Cawthorn, A., and Ford, W. (1994). Abalone. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C55C7W.

[Shmueli, 2010] Shmueli, G. (2010). To Explain or to Predict? *Statistical Science*, 25(3):289 – 310.

[Strobl et al., 2008] Strobl, C., Boulesteix, A.-L., Kneib, T., Augustin, T., and Zeileis, A. (2008). Conditional variable importance for random forests. *BMC bioinformatics*, 9:307.

[Vovk et al., 2005] Vovk, V., Gammerman, A., and Shafer, G. (2005). *Algorithmic Learning in a Random World*.

[Yeh, 1998] Yeh, I.-C. (1998). Concrete Compressive Strength. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5PK67.

[Zhao and Hastie, 2021] Zhao, Q. and Hastie, T. (2021). Causal interpretations of black-box models. *Journal of Business & Economic Statistics*, 39(1):272–281.